



Análisis Predictivo: Maratón de Boston (2015-2017)

Asignatura: Aprendizaje Automático

Alba Martínez de la Hermosa

Daniel Lopez Paredes

Alonso González Romero

11 de May, 2026

Table of contents

1	Introducción	4
	Contextualización en el dominio deportivo	4
	Justificación e interés del problema	4
	Objetivos del estudio	4
	Objetivo general	5
	Objetivos específicos	5
	Estructura del proyecto	5
2	Data Preprocessing	6
2.1	Importación de librerías	6
2.2	Carga de datos	6
2.3	Estructura del conjunto de datos	6
2.3.1	Datos Sociodemográficos y de Origen	7
2.3.2	Tiempos de Parciales (<i>Splits</i>)	8
2.3.3	Rendimiento y Clasificación Final	8
2.4	Integración y Consolidación del Conjunto de Datos	9
2.5	Preprocesamiento y Limpieza de Variables	10
2.6	Análisis de datos nulos	11
2.7	Incorporación de Variables Meteorológicas	12
2.8	Categorización por Nivel de Rendimiento	12
2.9	Definición de la Variable de Fatiga Extrema (<i>Hitting the Wall</i>)	13
3	Análisis Exploratorio de Datos	15
3.1	Importación de librerías	15
3.2	Carga de datos	15
3.3	Análisis Univariante	15
3.3.1	Distribución de las variables objetivo (<i>hit_the_wall</i> y <i>official_time</i>)	15
3.3.2	Distribución de las variables Edad y Género	16
3.3.3	Top 10 países	17
3.4	Análisis Bivariado y Multivariado	17
3.4.1	Tiempo final vs Edad vs Género	17
3.4.2	Cantidad de corredores en cada categoría	18
3.4.3	Impacto del clima	19
3.4.4	Muro por categorías	20
4	Aprendizaje No Supervisado	22
4.1	Clustering No Jerárquico: Identificación de Estrategias de Carrera (<i>Pacing</i>)	22
4.1.1	Fundamento Teórico: El Algoritmo K-Means	22
4.1.2	Metodología de Ejecución y Análisis	23
4.2	Clustering Jerárquico: Análisis Táctico de la Élite Femenina (2015)	28
4.2.1	Fundamento Teórico: Enfoque Aglomerativo (<i>Bottom-Up</i>)	28
4.2.2	Análisis Táctico	29

4.3	Clustering en Streaming: Simulación de Carrera en Vivo	31
4.3.1	Fundamentación teórica	31
4.3.2	Clustering con grupos preestablecidos	32
4.3.3	Clustering sin grupos preestablecidos	33
5	Aprendizaje Supervisado	36
5.1	Problema de clasificación. Detección de atletas que sufren en el muro.	36
5.1.1	K-Vecinos Más Cercanos (K-NN)	37
5.1.2	Árbol de Decisión	39
5.1.3	Random Forest	42
5.1.4	Selección del mejor modelo y predicción en el conjunto de Validación . . .	44
5.2	Problema de regresión. Predicción de tiempo final según sus datos hasta el parcial 35km	47
5.2.1	Planteamiento del problema y selección de variables	47
5.2.2	Distribución de la variable objetivo	48
5.2.3	Preprocesamiento para modelos lineales	50
5.2.4	Métricas de evaluación en regresión	50
5.2.5	GLM 1: Regresión Lineal	52
5.2.6	GLM 2: Ridge Regression	52
5.2.7	GLM 3: Lasso Regression	53
5.2.8	Interpretación de coeficientes	55
5.2.9	Selección del mejor GLM y validación final	56
5.2.10	Análisis de errores en validación	57
6	Discusion/Conclusiones	62

1 Introducción

Contextualización en el dominio deportivo

El maratón es una prueba de resistencia de 42,195 kilómetros que representa uno de los retos de gestión energética y fisiológica más complejos en el ámbito del deporte. Es consolidada como una disciplina olímpica tanto masculina como femenina, exigiendo a los atletas un control exhaustivo de su biomecánica y táctica de carrera. En una prueba de esta magnitud, el éxito depende de la eficiencia metabólica y de una precisa distribución del esfuerzo a lo largo del tiempo (pacing).

Para llevar a cabo este estudio, se han seleccionado los registros históricos oficiales del Maratón de Boston en sus ediciones consecutivas de 2015, 2016 y 2017. La evolución tecnológica en el cronometraje deportivo, específicamente el uso de chips de seguimiento individuales, permite registrar con total precisión los tiempos transcurridos acumulados desde la salida hasta diferentes puntos de control estratégico. Gracias a esta monitorización, disponemos de los tiempos de paso parciales de cada corredor cada 5 kilómetros y en la media maratón. Esta disponibilidad de datos granulares transforma una carrera continua en una sucesión de sectores analizables, abriendo la puerta a entender la dinámica de la prueba durante su transcurso y a ampliar el contexto disponible sobre los resultados finales obtenidos.

Justificación e interés del problema

En el ámbito del Sport Analytics, el estudio de un maratón abarca múltiples frentes de gran valor práctico: desde descifrar las mejores estrategias de ritmo (pacing) para optimizar el esfuerzo en futuras ediciones, hasta reconstruir la propia dinámica y evolución táctica durante el transcurso de la prueba. Asimismo, el desarrollo de herramientas predictivas capaces de estimar el tiempo final en meta o de alertar sobre un colapso fisiológico inminente proporciona un soporte fundamental para que los entrenadores puedan ajustar la toma de decisiones en tiempo real.

La unificación de los registros históricos de 2015, 2016 y 2017 amplifica enormemente el interés y la solidez de este análisis. Al tratarse del Maratón de Boston, uno de los eventos más prestigiosos del circuito World Marathon Majors, se garantiza el trabajo con una muestra de datos de altísima calidad, compuesta por atletas con una notable disciplina y preparación.

Objetivos del estudio

Este trabajo establece dos objetivos principales: la identificación de patrones tácticos subyacentes y la anticipación del rendimiento futuro.

Objetivo general

Desarrollar y validar modelos de Aprendizaje Automático, tanto supervisados como no supervisados, capaces de descifrar las estrategias de carrera (pacing) y predecir métricas críticas de rendimiento en el Maratón de Boston, integrando datos sociodemográficos, tiempos de paso parciales y condiciones meteorológicas.

Objetivos específicos

1. **Análisis Táctico y de Pacing:** Descubrir y categorizar de forma automática las diferentes estrategias de distribución del esfuerzo empleadas por los corredores mediante algoritmos de clustering, tanto jerárquico como no jerárquico.
2. **Estudio del transcurso de la carrera en atletas de Élite:** Diseccionar el comportamiento táctico del pelotón de élite femenina mediante clustering en streaming con el objetivo de entender la narrativa de la carrera.
3. **Detección Temprana del “Muro”:** Construir un modelo predictivo capaz de clasificar qué atletas sufrirán una deceleración severa en la segunda mitad de la prueba, basándose en los datos conocidos en la primera parte de la carrera.
4. **Predicción de Marca Final:** Evaluar y seleccionar el modelo de regresión más preciso para estimar el tiempo oficial en meta de un corredor al paso por el kilómetro 35, optimizando la toma de decisiones en el tramo final.

Estructura del proyecto

Para responder a los objetivos planteados, el presente informe se estructura en cuatro partes. Primero, se lleva a cabo un preprocesamiento de los datos. A continuación, se desarrolla un Análisis Exploratorio de Datos para examinar posibles relaciones entre variables. Posteriormente, se aplica el Aprendizaje No Supervisado mediante técnicas de clustering para descubrir las estrategias de carrera (pacing) tanto del dataset en general como de las atletas de élite, además de realizar clustering en streaming con el objetivo de entender cómo se desarrolló la carrera. Finalmente, se realizan modelos de Aprendizaje Supervisado, abordando tanto la clasificación para la detección temprana de los atletas que colapsan contra el “muro”, como la regresión para proyectar con precisión la marca definitiva en meta al paso por el kilómetro 35.

2 Data Preprocessing

En este primer capítulo se realiza la carga inicial de los datos, descripción inicial del dataset, limpieza de datos, tratamiento de valores faltantes y creación de nuevas variables externas sobre el clima del día de la carrera que puedan resultar significativas para el posterior entrenamiento de los modelos.

2.1 Importación de librerías

Para comenzar, cargamos el ecosistema básico de librerías en Python para realizar dicho trabajo de exploración y preprocesamiento.

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
import datetime
```

2.2 Carga de datos

Disponemos de tres archivos independientes en formato CSV que contienen los resultados oficiales de la Maratón de Boston para las ediciones de 2015, 2016 y 2017. Procedemos a su lectura para poder posteriormente empezar a realizar los diferentes análisis.

```
df_2015 = pd.read_csv('../data/raw/results2015.csv')
df_2016 = pd.read_csv('../data/raw/results2016.csv')
df_2017 = pd.read_csv('../data/raw/results2017.csv')
```

2.3 Estructura del conjunto de datos

Una vez tenemos los tres conjuntos de datos, vamos a realizar un pequeño estudio de la estructura que tienen, para ello, vamos a comenzar revisando si tienen exactamente las mismas variables:

```
# Comprobamos si las columnas y su orden son idénticos
todo_igual = (df_2015.columns.equals(df_2016.columns) and
df_2016.columns.equals(df_2017.columns))

# Obtenemos el número de columnas de cada dataframe
num_cols_15 = len(df_2015.columns)
```

```
num_cols_16 = len(df_2016.columns)
num_cols_17 = len(df_2017.columns)

# Imprimimos el resultado
if todo_igual:
    print(f"¿Son exactamente iguales en columnas y orden?:"
          f"Sí, y además tienen {num_cols_15} columnas.")
else:
    print(f"¿Son exactamente iguales en columnas y orden?:"
          f"No, tienen {num_cols_15}, {num_cols_16} y {num_cols_17} "
          f"columnas respectivamente.")
```

¿Son exactamente iguales en columnas y orden?:Sí, y además tienen 29 columnas.

Por lo tanto, tenemos 3 conjuntos de datos iguales y que además tienen 29 columnas.

El dataset original de la Maratón de Boston (ediciones 2015, 2016 y 2017) cuenta con un conjunto de variables que combinan datos sociodemográficos, tiempos de paso parciales y clasificaciones finales. A continuación se detalla el significado de cada columna estructurado por categorías:

2.3.1 Datos Sociodemográficos y de Origen

Variable	Tipo de Dato	Descripción
bib	string	Número de dorsal del corredor. <i>Nota:</i> Puede contener caracteres no numéricos en atletas de élite o categorías especiales (ej. F1, W10).
name	Object	Nombre completo del atleta. Existen otras 2 columnas que dividen este nombre en nombre y apellidos
age	float64	Edad del corredor el día de la carrera.
gender	string	Género del corredor (M para masculino, F para femenino).
city	string	Ciudad de residencia registrada por el participante.
state	string	Estado o provincia de residencia (principalmente para corredores de EE. UU. y Canadá).

Variable	Tipo de Dato	Descripción
<code>country_residence</code>	<code>string</code>	Código de 3 letras que representa el país de origen del corredor (ej. USA, ESP, MEX).
<code>contry_citizenship</code>	<code>string</code>	Nacionalidad o país de ciudadanía del corredor (suele estar vacío si coincide con el país de residencia).

2.3.2 Tiempos de Parciales (*Splits*)

Estas columnas registran el tiempo transcurrido acumulado desde el pistoletazo de salida (o cruce de la línea de salida) hasta cada punto de control específico.

Variable	Tipo de Dato	Punto de Control	Distancia Acumulada
5K	<code>string</code>	Tiempo de paso por el kilómetro 5.	5,0 km
10K	<code>string</code>	Tiempo de paso por el kilómetro 10.	10,0 km
15K	<code>string</code>	Tiempo de paso por el kilómetro 15.	15,0 km
20K	<code>string</code>	Tiempo de paso por el kilómetro 20.	20,0 km
Half	<code>string</code>	Tiempo de paso por la Media Maratón.	21,097 km
25K	<code>string</code>	Tiempo de paso por el kilómetro 25.	25,0 km
30K	<code>string</code>	Tiempo de paso por el kilómetro 30.	30,0 km
35K	<code>string</code>	Tiempo de paso por el kilómetro 35.	35,0 km
40K	<code>string</code>	Tiempo de paso por el kilómetro 40.	40,0 km

2.3.3 Rendimiento y Clasificación Final

Variable	Tipo de Dato	Descripción
<code>Pace</code>	<code>string</code>	Ritmo medio por milla de toda la carrera en formato MM:SS.

Variable	Tipo de Dato	Descripción
<code>projected_time</code>	string	Tiempo final estimado o proyectado (generalmente equivalente al tiempo oficial).
<code>official_time</code>	Object	Tiempo neto oficial de meta (Chip Time). Representa el tiempo real de carrera desde que el atleta cruza la línea de salida hasta que pisa la de meta. (Variable Objetivo / Target Y).
<code>overall</code>	Integer	Posición en la clasificación general absoluta de la carrera.
<code>gender_result</code>	Integer	Posición en la clasificación general de su respectivo género.
<code>division_result</code>	Integer	Posición en la clasificación de su grupo de edad específico.
<code>seconds</code>	Integer	Segundos tardados en completar la maratón

2.4 Integración y Consolidación del Conjunto de Datos

Para poder entrenar y validar nuestros modelos de aprendizaje automático de forma global, es necesario unificar las tres fuentes de datos en un único dataframe consolidado.

Antes de realizar la concatenación, añadiremos la variable **Date** con la fecha exacta de realización de cada edición. La Maratón de Boston se celebra históricamente el tercer lunes de abril (Patriots' Day), correspondiendo a las siguientes fechas reales:

- **Edición 2015:** 20 de abril de 2015 (2015-04-20)
- **Edición 2016:** 18 de abril de 2016 (2016-04-18)
- **Edición 2017:** 17 de abril de 2017 (2017-04-17)

Al añadir esta variable temporal no solo mantenemos la trazabilidad del año de origen de cada registro, sino que habilitamos la posibilidad de cruzar estos datos en el futuro con variables externas como la meteorología del día exacto del evento.

```
# 1. Añadimos la columna 'Date' a cada dataframe individual en formato string
df_2015['Date'] = '2015-04-20'
df_2016['Date'] = '2016-04-18'
df_2017['Date'] = '2017-04-17'

# 2. Concatenamos los tres conjuntos de datos verticalmente
```

```
df_boston = pd.concat([df_2015, df_2016, df_2017], ignore_index=True)

# 3. Convertimos la columna 'Date' a tipo datetime oficial de Pandas
df_boston['Date'] = pd.to_datetime(df_boston['Date'])

# 4. Verificamos las dimensiones finales del dataset unificado
print(f"Dimensiones del dataset unificado final: {df_boston.shape}")
print(f"Número total de registros: {df_boston.shape[0]} corredores.")
print(f"Número total de columnas: {df_boston.shape[1]}")
```

Dimensiones del dataset unificado final: (79640, 30)
Número total de registros: 79640 corredores.
Número total de columnas: 30

Una vez realizada la unificación, verificamos los datos realizando un muestreo de algunas columnas:

```
# Seleccionamos una muestra de columnas
columnas_muestra = ['name', 'age', 'gender', 'country_residence',
                    'official_time', 'Date']
df_muestra = df_boston[columnas_muestra].sample(5, random_state=42)

# Imprimimos la muestra formateada en markdown limpio
print(df_muestra.to_markdown(index=False))
```

name	age	gender	country_residence	official_time	Date
Loaiza, David	48	M	USA	3:24:27	2017-04-17 00:00:00
Fulfer, Clay	52	M	USA	5:14:15	2016-04-18 00:00:00
Meadows, Karen L	52	F	USA	4:00:58	2016-04-18 00:00:00
Johnson, David T	32	M	USA	2:54:17	2015-04-20 00:00:00
Paine, A. R.	51	M	USA	3:53:52	2016-04-18 00:00:00

2.5 Preprocesamiento y Limpieza de Variables

Una vez consolidado nuestro conjunto de datos, es fundamental adecuar los tipos de datos de las variables y descartar aquellas columnas redundantes o irrelevantes (como identificadores de nombres alternativos, sufijos o formatos duplicados) antes de alimentar nuestros algoritmos de Aprendizaje Automático.

En esta fase de preprocesamiento realizaremos las siguientes acciones:

1. **Selección de características (*Feature Selection*):** Conservaremos únicamente el subconjunto de variables demográficas, espaciales y de rendimiento necesarias, descartando variables de texto libre como `name` o redundantes como `seconds`.
2. **Corrección de tipos de datos:** Convertiremos la edad (`age`) de formato decimal a entero, y las variables de texto repetitivo como `gender` y `country_residence` a tipo categórico.
3. **Conversión temporal (Formatos HH:MM:SS a segundos):** Transformaremos todos los parciales de paso (5k, 10k, etc.) y la variable objetivo (`official_time`) a segundos totales, permitiendo que los modelos matemáticos puedan procesar estas distancias como variables numéricas continuas.
4. **Tratamiento de la fecha:** Simplificaremos la columna `Date` para conservar únicamente la fecha del evento sin la hora.

Tipos de datos finales del dataset preprocesado:

```

bib                object
display_name       object
age                int64
gender              category
city                object
country_residence  category
5k                  float64
10k                 float64
15k                 float64
20k                 float64
half                float64
25k                 float64
30k                 float64
35k                 float64
40k                 float64
pace                object
official_time       int64
overall             float64
gender_result       float64
division_result     float64
Date                object
dtype: object

```

2.6 Análisis de datos nulos

Una vez tenemos todos los datos enriquecidos con su formato ideal y las variables necesarias para realizar los modelos, ahora se va a revisar qué datos nulos tenemos y en qué variables. Para ello, comenzamos viendo en qué variables hay datos nulos y el número:

Variable	Valores Nulos	Total Registros	Nulos (%)
5k	229	79638	0.29
10k	114	79638	0.14
15k	51	79638	0.06
20k	85	79638	0.11

Variable	Valores Nulos	Total Registros	Nulos (%)
half	62	79638	0.08
25k	81	79638	0.1
30k	88	79638	0.11
35k	86	79638	0.11
40k	76	79638	0.1

Como se puede observar en la tabla resumen, la presencia de valores faltantes es estadísticamente marginal, representando un máximo del 0.29% de los registros en el peor de los casos.

Desde una perspectiva metodológica, ante la falta de registros en puntos de control intermedios, se podría haber planteado una estrategia de imputación algorítmica. Específicamente, en aquellos atletas que presentaran únicamente uno o dos parciales faltantes no consecutivos, resultaría viable aplicar un modelo de K-Vecinos Más Cercanos (K-NN) para estimar el tiempo perdido basándose en la trayectoria de los corredores más afines en los sectores previos y posteriores.

No obstante, se ha decidido descartar esta aproximación y proceder a la eliminación directa de las filas afectadas. Esta decisión se fundamenta en el gran volumen de datos, en la que la pérdida de unas cuantas observaciones no compromete la representatividad de la muestra.

2.7 Incorporación de Variables Meteorológicas

El rendimiento en una maratón está fuertemente condicionado por factores ambientales externos. Para que nuestros modelos de aprendizaje automático puedan capturar este impacto, añadiremos tres variables climatológicas clave asociadas a la fecha de cada edición:

1. **temp_mean_c**: Temperatura promedio durante las horas de la carrera (en grados Celsius).
2. **humidity_pct**: Porcentaje de humedad relativa promedio.
3. **wind_speed_kmh**: Velocidad promedio del viento (factor crítico en Boston debido a su recorrido lineal, donde el viento de cara actúa como un freno constante).

Las condiciones registradas para cada edición fueron:

Date	temp_mean_c	humidity_pct	wind_speed_kmh
2015-04-20	8	93	22
2016-04-18	16	28	13
2017-04-17	21	35	11

2.8 Categorización por Nivel de Rendimiento

Con el objetivo de enriquecer el análisis descriptivo y facilitar posteriores estrategias de muestreo estratificado o validación cruzada controlada, procedemos a crear una nueva variable categórica ordinal que clasifique el nivel de rendimiento de cada atleta.

Para definir los umbrales de rendimiento, adaptamos el criterio oficial de la World Athletics y los tiempos de corte de los cajones de salida de las *World Marathon Majors* (como Boston). Dado

que los ritmos basales difieren fisiológicamente por sexo, establecemos reglas diferenciadas para hombres (M) y mujeres (F):

- **Élite:**
 - Hombres: $< 2\text{h } 17\text{m } 00\text{s}$ (< 8220 segundos)
 - Mujeres: $< 2\text{h } 43\text{m } 00\text{s}$ (< 9780 segundos)
- **Alto nivel:**
 - Hombres: $[2\text{h } 17\text{m } 00\text{s} - 2\text{h } 30\text{m } 00\text{s})$ (8220 a < 9000 segundos)
 - Mujeres: $[2\text{h } 43\text{m } 00\text{s} - 3\text{h } 00\text{m } 00\text{s})$ (9780 a < 10800 segundos)
- **Muy entrenado/a:**
 - Hombres: $[2\text{h } 30\text{m } 00\text{s} - 3\text{h } 00\text{m } 00\text{s})$ (9000 a < 10800 segundos)
 - Mujeres: $[3\text{h } 00\text{m } 00\text{s} - 3\text{h } 30\text{m } 00\text{s})$ (10800 a < 12600 segundos)
- **Moderadamente entrenado/a:**
 - Hombres: $[3\text{h } 00\text{m } 00\text{s} - 3\text{h } 30\text{m } 00\text{s})$ (10800 a < 12600 segundos)
 - Mujeres: $[3\text{h } 30\text{m } 00\text{s} - 4\text{h } 00\text{m } 00\text{s})$ (12600 a < 14400 segundos)
- **Principiante:**
 - Hombres: $\geq 3\text{h } 30\text{m } 00\text{s}$ (≥ 12600 segundos)
 - Mujeres: $\geq 4\text{h } 00\text{m } 00\text{s}$ (≥ 14400 segundos)

A continuación, implementamos una función de clasificación y la aplicamos sobre nuestro dataset unificado.

athlete_level	Número de Corredores	Porcentaje (%)
Principiante	39845	50.41
Moderadamente entrenado/a	28452	36
Muy entrenado/a	10262	12.98
Alto nivel	393	0.5
Élite	86	0.11

Por lo tanto, se observa que la lógica aplicada tiene sentido según el porcentaje de corredores que hay en cada umbral.

2.9 Definición de la Variable de Fatiga Extrema (*Hitting the Wall*)

Para habilitar un posterior enfoque de **Aprendizaje Supervisado de Clasificación**, definimos una nueva variable objetivo binaria denominada `hit_the_wall` (coloquialmente conocido como “petar”).

Establecemos el umbral de fatiga siguiendo los estándares de la literatura científica deportiva: un atleta ha sufrido una deceleración crítica si el tiempo empleado en completar la segunda mitad de la maratón es igual o superior al **120% (un 20% más lento)** del tiempo invertido en la primera mitad.

- **Primera Mitad (T_1):** Tiempo de paso por la media maratón (`half`).

- **Segunda Mitad (T_2):** Official Time – half.
- **Condición de “Petar”:** $T_2 \geq 1.20 \times T_1 \implies \text{hit_the_wall} = 1$.

	Número de Corredores	Porcentaje (%)
Ritmo Constante (0)	63060	79.78
Chocan contra el muro (1)	15978	20.22

3 Análisis Exploratorio de Datos

Una vez finalizada la fase de preprocesamiento e ingeniería de variables, disponemos de un conjunto de datos consolidado, depurado y enriquecido. El objetivo de este capítulo es desarrollar un Análisis Exploratorio de Datos (EDA) estructurado que nos permita comprender más acerca de los datos que tenemos y la relación entre sus distintas variables.

3.1 Importación de librerías

Comenzamos importando las librerías necesarias para esta sección

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
from scipy import stats
```

3.2 Carga de datos

Procedemos a la lectura del archivo de datos unificado de las 3 ediciones junto con todos los cambios realizados en el capítulo de preprocesado.

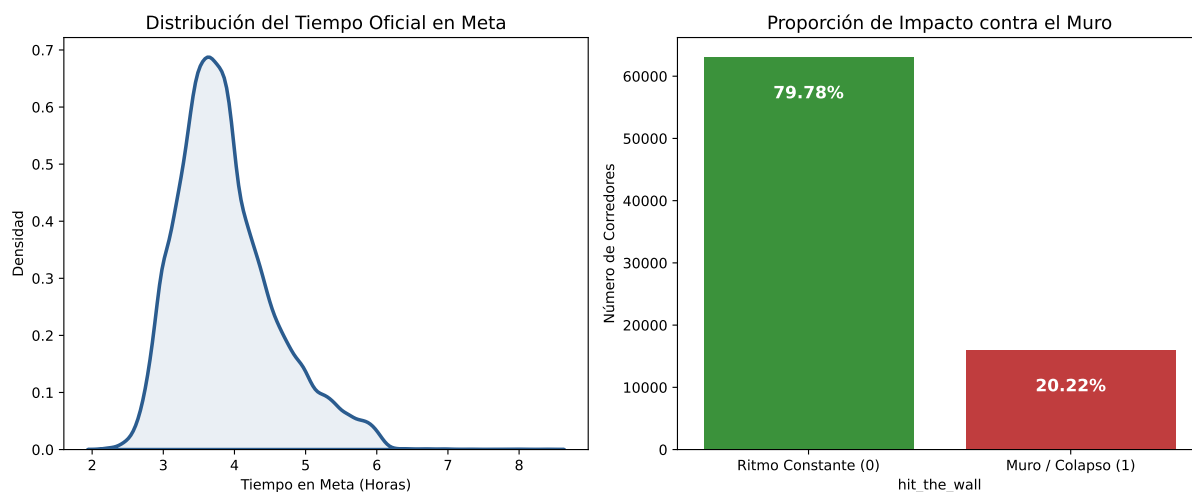
```
df_boston = pd.read_parquet("../data/processed/boston_post_eda.parquet")
```

3.3 Análisis Univariante

El objetivo de esta sección es comprender mejor la distribución de las variables por separado. Para ello, comenzamos revisando la distribución de las dos variables objetivo.

3.3.1 Distribución de las variables objetivo (`hit_the_wall` y `official_time`)

Nuestro estudio aborda dos enfoques predictivos distintos: un problema de regresión sobre el tiempo final en meta (`official_time`) y un problema de clasificación binaria para la detección del colapso fisiológico (`hit_the_wall`). A continuación, inspeccionamos la composición y distribución de ambas metas.

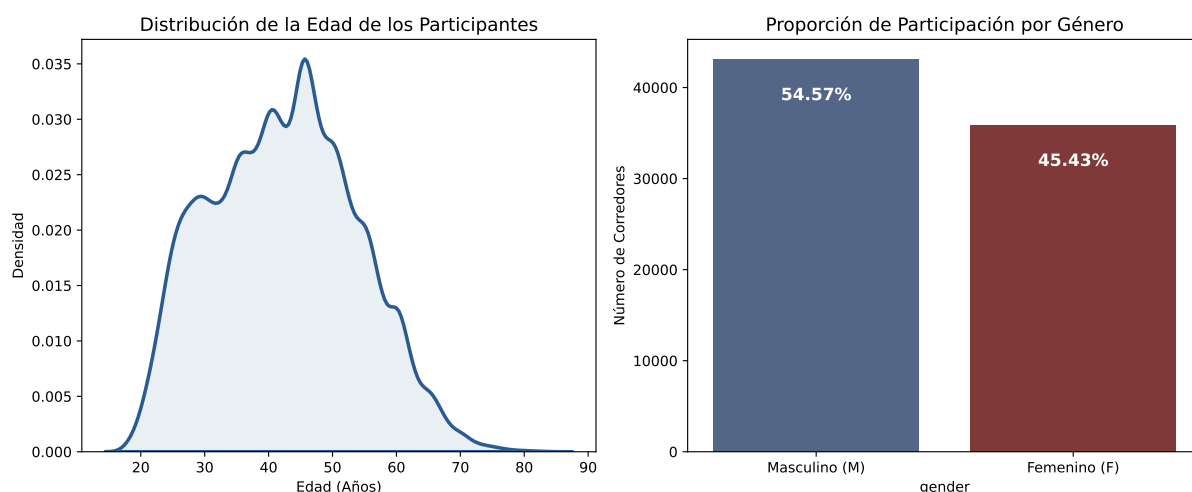


La distribución del tiempo oficial presenta una clara asimetría positiva, concentrando a la gran mayoría del pelotón entre las 3 horas y cuarto y las 4 horas, lo que evidencia el alto nivel de exigencia y la criba previa de marcas que requiere esta competición.

Por su parte, la variable del muro muestra que el 79.78% de los atletas logran mantener un ritmo constante, mientras que el 20.22% colapsan y sufren una deceleración severa en la segunda mitad de la prueba. Esta proporción confirma que nos enfrentamos a un problema de clasificación desbalanceado, un factor clave que deberá tenerse en cuenta al configurar los pesos en los modelos predictivos posteriores.

3.3.2 Distribución de las variables Edad y Género

En este apartado, vamos a visualizar la distribución de las variables de Edad y Género:



El perfil demográfico de los participantes refleja una población madura y equilibrada, con una mayoría de corredores situados en torno a los 45 años, lo que evidencia la experiencia necesaria para alcanzar los tiempos de calificación exigidos en Boston. En cuanto al género, la muestra presenta una paridad notable con un 54.57% de hombres frente a un 45.43% de mujeres, una distribución muy positiva para el entrenamiento de los posteriores modelos.

3.3.3 Top 10 países

Con el objetivo de visualizar las nacionalidades de los atletas, a continuación vamos a identificar las 10 naciones con mayor volumen de participación.

País	Número de Corredores	Porcentaje (%)
USA	64001	80.97
CAN	6135	7.76
GBR	1063	1.34
MEX	757	0.96
GER	569	0.72
JPN	488	0.62
AUS	469	0.59
ITA	466	0.59
CHN	427	0.54
BRA	425	0.54

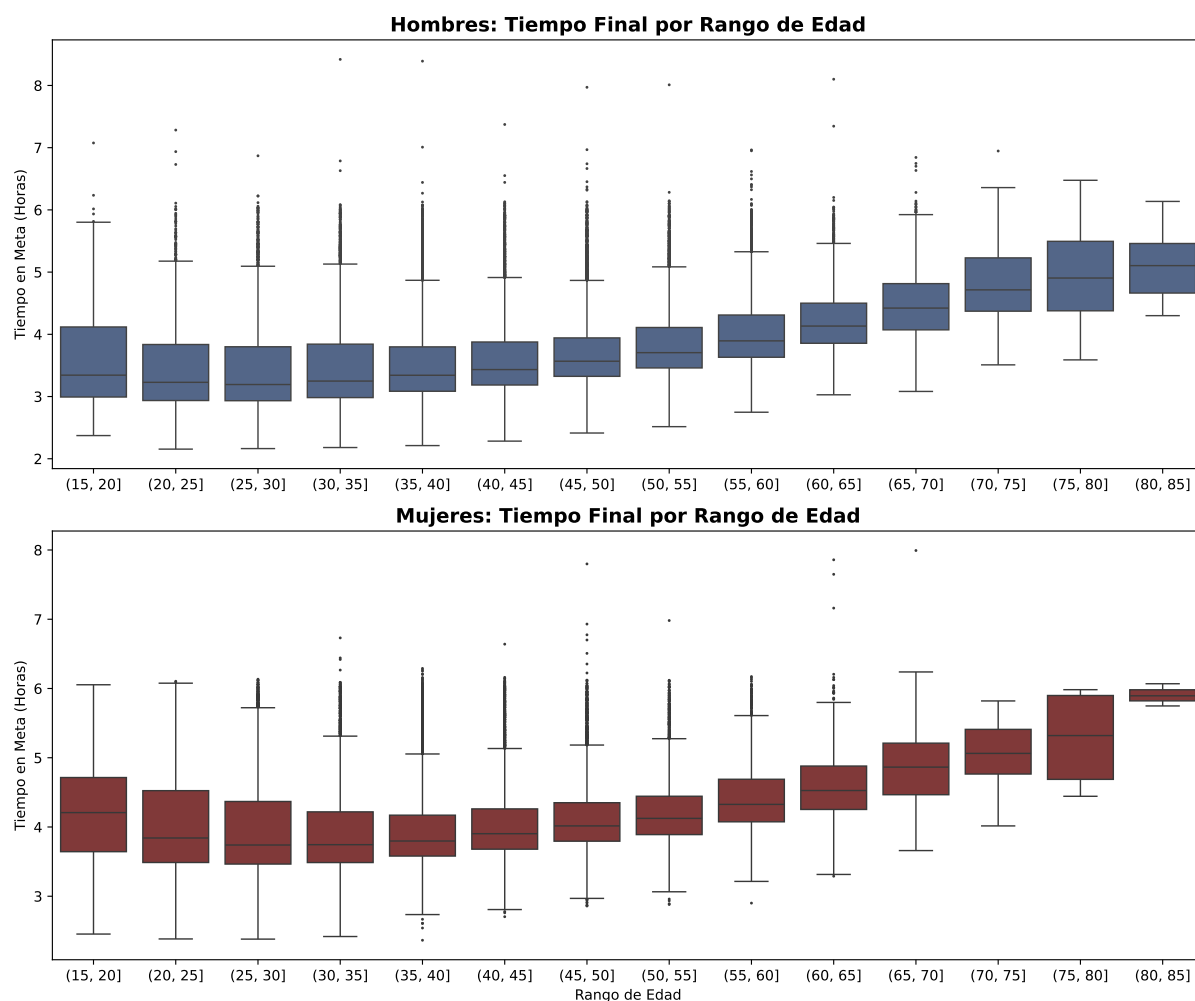
La tabla refleja que EEUU es el país que más participantes concentra, casi el 81% de los participantes. De cara al entrenamiento de los modelos, manejar una variable con tantos países distintos puede complicar el aprendizaje del algoritmo debido a la gran cantidad de categorías con muy pocos datos. Por ello, una solución eficaz para mejorar la precisión será simplificar esta información en la fase de modelado, ya sea creando una variable binaria (EE. UU. frente al Resto del mundo) o agrupando los países por continentes, facilitando así que el modelo identifique patrones geográficos de rendimiento de manera más clara y robusta.

3.4 Análisis Bivariado y Multivariado

El objetivo de esta sección es identificar patrones y relaciones entre variables a través de un análisis bivariado y multivariado.

3.4.1 Tiempo final vs Edad vs Género

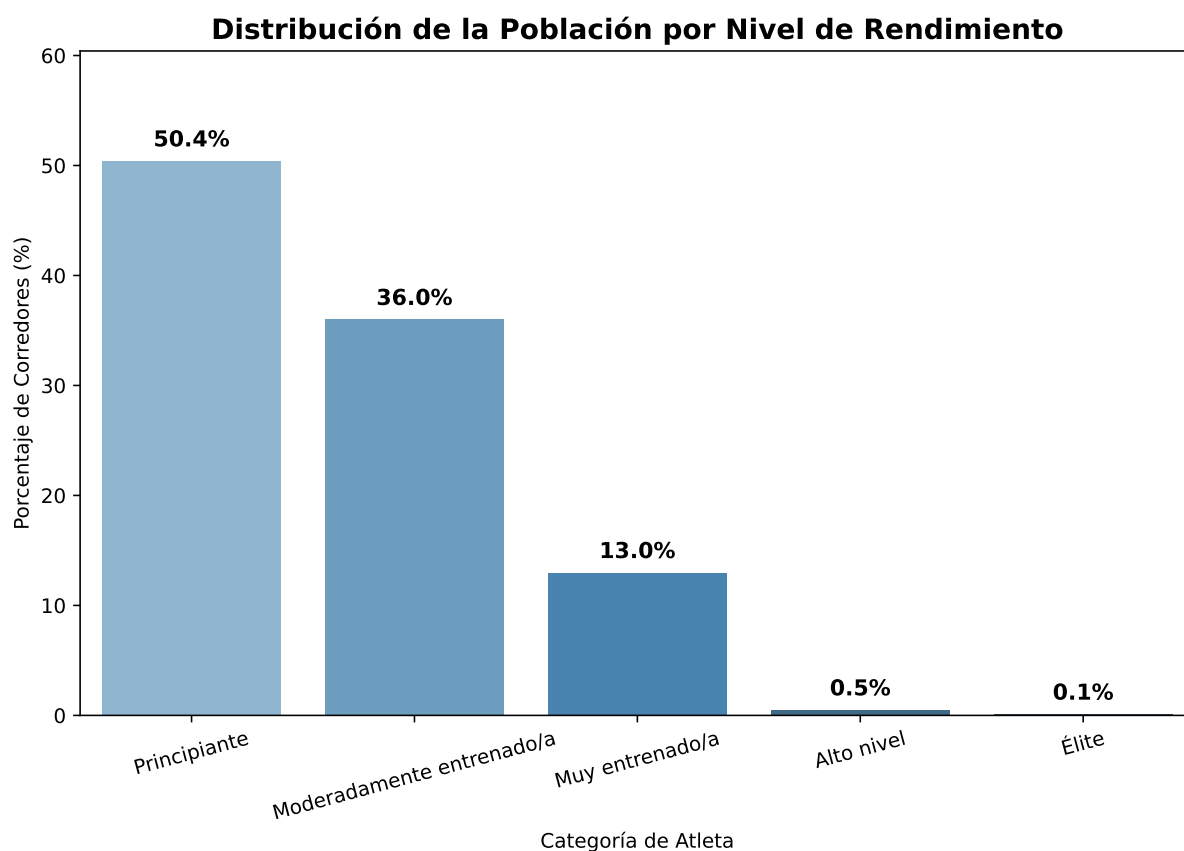
A continuación, vamos a comprobar las distribuciones del tiempo tardado en la maratón en intervalos de 5 años de edad para ver cuál es el mejor rango de edad en cuanto a rendimiento. Además, vamos a agrupar dichos resultados por género para obtener una gráfica distinta para cada uno.



El rendimiento en la maratón se mantiene increíblemente estable hasta los 50 años en ambos sexos, situándose la mediana de tiempo ligeramente por encima de las 3 horas en hombres y en torno a las 3 horas y media en mujeres. Es a partir de la franja de 50-55 años cuando se observa un punto de inflexión, donde los tiempos medios comienzan a subir de forma progresiva.

3.4.2 Cantidad de corredores en cada categoría

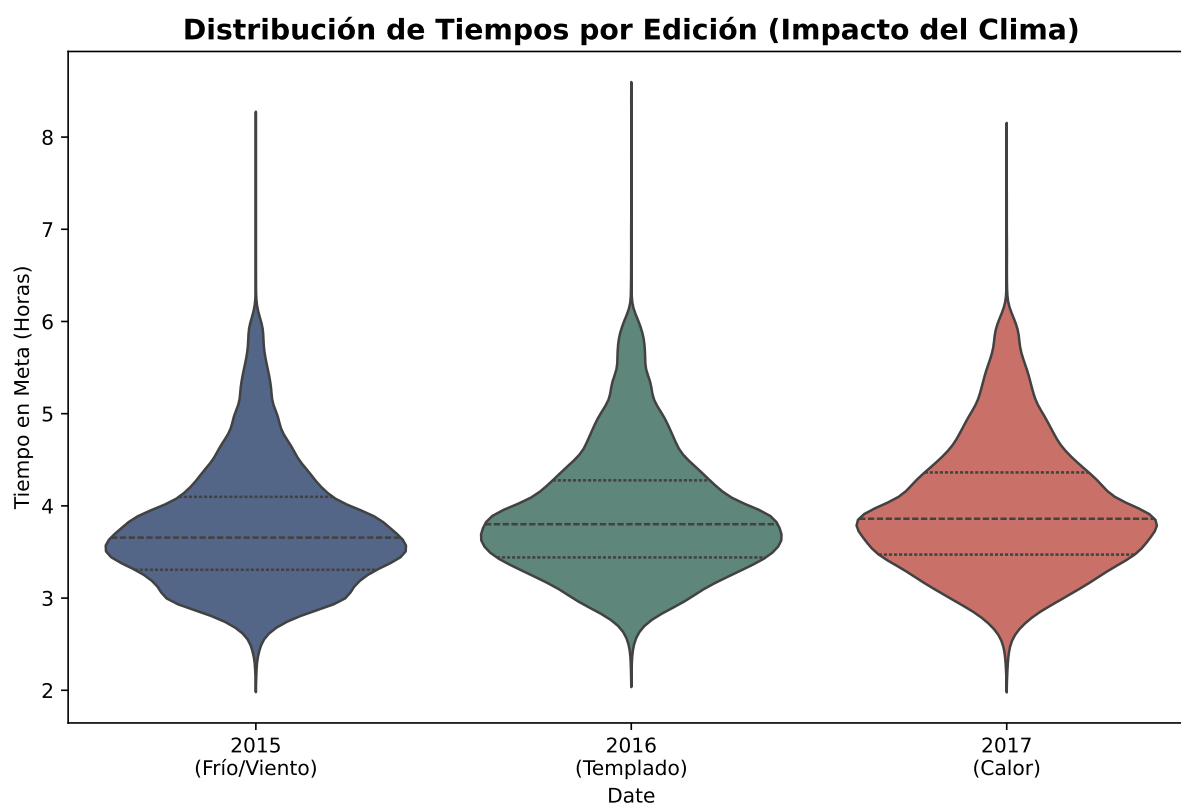
En esta sección, se el objetivo será graficar cuántos corredores hay en cada categoría



La categorización de los atletas por nivel de rendimiento muestra una estructura piramidal muy clara. La mitad del pelotón (50.4%) se clasifica en el umbral de Principiante (marcas superiores a 3h 30m en hombres y 4h en mujeres), seguido de un 36.0% de corredores Moderadamente entrenados. Por su parte, la élite y los atletas de alto nivel representan una fracción muy baja, de apenas el 0.6% sumando ambos.

3.4.3 Impacto del clima

Una vez revisados la cantidad de corredores por categoría, en este apartado se va a evaluar la distribución de tiempos por año.



Se observa que a medida que la edición avanza, los tiempos en media parecen ser mayores. Para confirmar esto, se va a realizar un test estadístico sobre la categoría *Élite*, ya que son los únicos corredores en los que se puede obtener una medida fiable de si el recorrido era más rápido o lento respecto al clima de esa edición

Date	Media_Horas	Desviacion	N
2015-04-20	2.372	0.174	29
2016-04-18	2.471	0.171	21
2017-04-17	2.448	0.185	36

****Resultado del test ANOVA en *Élite*:****

- Estadístico F: 2.2921

- P-valor: 1.0742e-01

Por lo que se puede observar en la tabla y en el p-valor, no existen diferencias significativas en sus tiempos medios a lo largo de las 3 ediciones.

3.4.4 Muro por categorías

Por último, vamos a evaluar el número de corredores que sufre el colapso en el muro según el nivel del atleta:

Categoría	Nº Corredores	Casos de Muro	% del Muro
Principiante	39845	13164	33.04
Moderadamente entrenado/a	28452	2570	9.03
Muy entrenado/a	10262	243	2.37
Alto nivel	393	1	0.25
Élite	86	0	0

A medida que aumenta el nivel de entrenamiento del atleta, la probabilidad de sufrir un colapso en la segunda mitad de la prueba se desploma drásticamente. Mientras que 1 de cada 3 corredores principiantes (33.04%) choca contra el muro —víctimas de una gestión del ritmo menos madura y de una menor adaptación al consumo eficiente de glucógeno—, esta cifra se reduce a un 9.03% en los moderadamente entrenados y cae a un marginal 2.37% en el grupo de muy entrenados.

Por su parte, en las categorías superiores el muro es prácticamente inexistente (0.25% en Alto nivel y 0% en la Élite). Estos datos confirman que el colapso fisiológico no es un evento puramente aleatorio de la distancia de maratón, sino una variable fuertemente predecible y dependiente de la preparación de los atletas.

4 Aprendizaje No Supervisado

En este capítulo se realizarán diversos estudios de Aprendizaje no Supervisado. Dada la naturaleza de los datos, se seguirán los siguientes enfoques:

1. **Clustering No Jerárquico** para descubrir las estrategias de carrera
2. **Clustering Jerárquico** para agrupar a corredores similares en cuanto a sus resultados en las 3 maratones
3. **Clustering en streaming** para seguir a los atletas de élite a través de cada parcial

4.1 Clustering No Jerárquico: Identificación de Estrategias de Carrera (*Pacing*)

El objetivo de esta sección es descubrir y categorizar de forma automática las diferentes estrategias de ritmo (*pacing strategies*) empleadas por los corredores durante la maratón. Para ello, utilizaremos técnicas de aprendizaje no supervisado, buscando agrupar a los atletas según la distribución de su esfuerzo a lo largo de los 42,195 km, independientemente de su marca final absoluta.

4.1.1 Fundamento Teórico: El Algoritmo K-Means

Para llevar a cabo esta agrupación, implementaremos el algoritmo **K-Means** (*K-Medias*), uno de los métodos de clustering particional más robustos y utilizados.

A diferencia del clustering jerárquico, K-Means requiere que se defina a priori el número de grupos (k) que se desean encontrar. El algoritmo funciona de manera iterativa siguiendo este proceso:

1. **Inicialización:** Se seleccionan aleatoriamente k puntos en el espacio de características que actuarán como los centros iniciales de los grupos (centroides, μ_i).
2. **Asignación:** Se calcula la distancia euclidiana entre cada corredor y todos los centroides. Cada corredor se asigna al clúster cuyo centroide esté más cercano.
3. **Actualización:** Una vez asignados todos los puntos, se recalcula la posición de cada centroide moviéndolo a la media matemática de todos los corredores que pertenecen a ese grupo.
4. **Convergencia:** Los pasos 2 y 3 se repiten de forma iterativa hasta que los centroides dejan de moverse o se alcanza un número máximo de iteraciones.

Matemáticamente, K-Means busca minimizar la varianza intra-clúster, conocida como **Inercia** (J), que se define como la suma de las distancias al cuadrado de cada punto respecto a su centroide:

$$J = \sum_{i=1}^k \sum_{x \in C_i} \|x - \mu_i\|^2$$

Donde k es el número de clústeres, C_i es el conjunto de puntos que pertenecen al clúster i , x es el punto de datos (la estrategia del corredor) y μ_i es el centroide del clúster.

4.1.2 Metodología de Ejecución y Análisis

Para que el algoritmo capture estrategias puras y no se sesgue por la velocidad absoluta del corredor, se debe seguir un riguroso proceso de ingeniería de características (*Feature Engineering*) y análisis posterior.

4.1.2.1 Transformación de los Tiempos (Normalización de Ritmos)

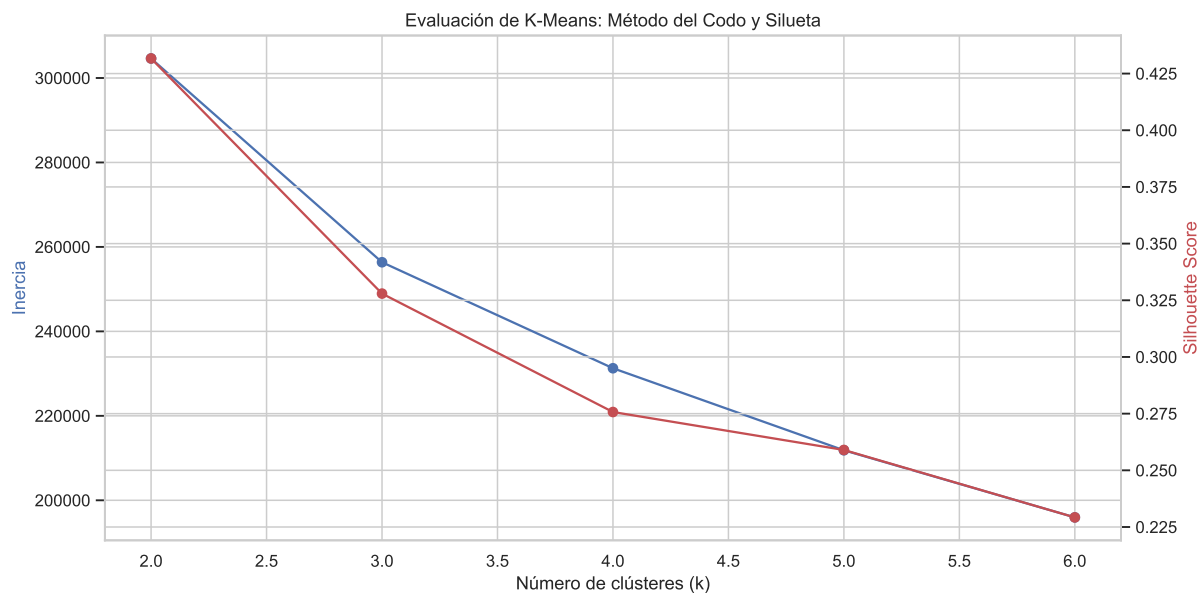
Los tiempos del dataset original son acumulativos. Para capturar el esfuerzo en cada tramo, se debe transformar la variable de paso calculando el **tiempo neto de cada segmento** (ej. $T_{10k} - T_{5k}$ para obtener el tiempo invertido exclusivamente entre el kilómetro 5 y el 10). Una vez calculados los tiempos netos por tramo, estos se deben dividir entre el tiempo final oficial (*official_time*). De este modo, cada variable representará el **porcentaje del tiempo total** que el atleta invirtió en ese parcial concreto.

split_1	split_2	split_3	split_4	split_5	split_6	split_7	split_8
11.3833	11.6024	11.7829	12.015	12.0923	12.2857	12.363	11.3
11.338	11.5562	11.7488	11.9414	12.057	12.2239	12.3267	11.3
11.2887	11.506	11.685	11.9151	11.9918	12.1836	12.2603	11.5
11.2527	11.4821	11.686	11.8262	11.9536	12.1448	12.2722	12.0
11.2498	11.4792	11.6448	11.8486	11.9633	12.1417	12.2181	11.8
11.2583	11.4748	11.6403	11.8441	11.946	12.137	12.2134	11.7
11.231	11.4213	11.8655	11.7132	11.7005	12.1066	12.2716	12.1
11.1027	11.3037	11.4921	11.693	11.7684	11.9568	12.0573	12.7
11.0168	11.2289	11.4161	11.6032	11.7155	11.8902	11.9775	12.4
11.3795	11.4542	11.5911	11.6783	11.6409	11.865	12.3506	12.3

4.1.2.2 Determinación del Número Óptimo de Clústeres (k)

Se iterará el algoritmo K-Means para distintos valores de k (por ejemplo, del 2 al 10). Para seleccionar el k óptimo que mejor defina las tipologías de corredores, se apoyará la decisión en dos métricas gráficas:

- **El Método del Codo (*Elbow Method*):** Analizando la curva de caída de la Inercia.
- **El Análisis de la Silueta (*Silhouette Score*):** Para medir la cohesión y separación real de los grupos formados.



ANÁLISIS DE CLUSTERING

Sugerencia por Silueta: 2

Sugerencia por Método del Codo: No detectado automáticamente.

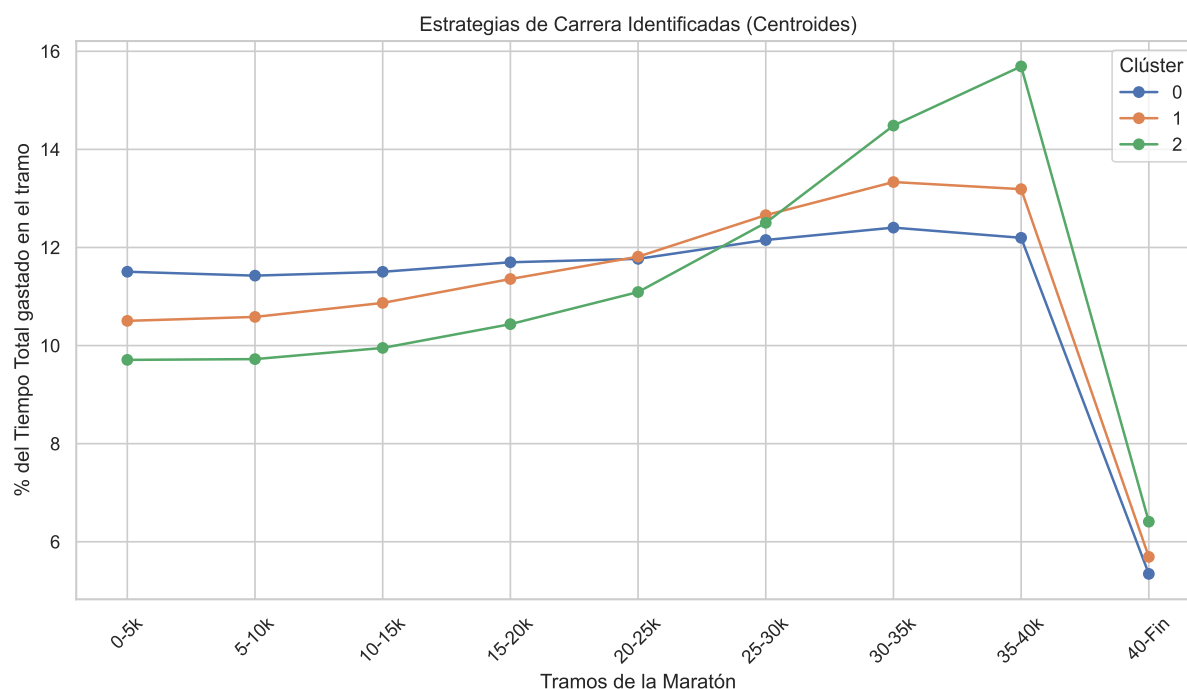
Conclusión: Basándonos solo en Silueta, el valor sugerido es $k=2$.

Revisa el gráfico para confirmar el número de clusters óptimo.

Aunque la recomendación inicial es de 2 clusters, al observar el gráfico se observa un cambio significativo en la pendiente de la gráfica en el número 3. En maratón, cambios que a priori parecen pequeños pueden marcar la diferencia entre el ganador y los demás, o entre conseguir el objetivo o no hacerlo. En pro de tener una visión granular de las estrategias de carrera se establece el número óptimo de clusters en 3.

4.1.2.3 Perfilado y Visualización de Estrategias

Una vez elegida la k óptima y asignados los grupos, se extraerán las coordenadas de los centroides finales. Se graficarán estos centroides en un gráfico de líneas superpuestas (Eje X: Segmento de carrera; Eje Y: Porcentaje de tiempo invertido). Esto permitirá nombrar a cada cluster.



En este apartado se diferencian 3 tipos de estrategia que tienen en común una bajada drástica de porcentaje de tiempo empleado desde el kilómetro 40 hasta la meta. Esto ocurre porque este tramo tiene solo 2195 metros frente a los 5km que forman todos los segmentos anteriores. Esto hace que el porcentaje de tiempo que los atletas emplean en recorrer ese segmento se reduzca a menos de la mitad.

En cuanto a los demás estadios de la carrera, se observa lo siguiente:

- La estrategia 0, dibujada de azul, es muy estable a lo largo de toda la carrera. No se ven variaciones significativas en el porcentaje de tiempo que los atletas emplean en cada uno de los segmentos. Este tipo de dinámica de carrera se conoce como **Uniforme**, caracterizada por mantener un ritmo constante; es el comportamiento óptimo para la maratón.
- La estrategia identificada con un 2 y dibujada de verde muestra un inicio más rápido (menos porcentaje de tiempo en los primeros estadios) y un final más lento (más porcentaje de tiempo en los segmentos de la mitad de la carrera hasta el final) con respecto a las otras estrategias. En otras palabras, la diferencia entre la velocidad inicial y final es considerablemente mayor. Esto refleja una bajada de rendimiento significativa desde la mitad de la carrera hasta el final, lo que se conoce como estrategia **Positiva**.
- La estrategia dibujada de naranja e identificada con el número 1 se ve estable aunque con ligeras fluctuaciones. Esta táctica sigue considerándose uniforme ya que dichas variaciones no son tan significativas como para afirmar que el final ha sido mucho más lento que el comienzo. Sin embargo, para diferenciarla de la estrategia uniforme pura, se le pondrá el nombre de **Uniforme-Positiva**.

4.1.2.4 Conclusiones y Cruce de Variables

Finalmente, se extraerá el valor de negocio de la agrupación realizando un análisis cruzado (*Crosstabulation*). Se evaluará, por ejemplo, la relación entre cada cluster y la variable `hit_the_wall`

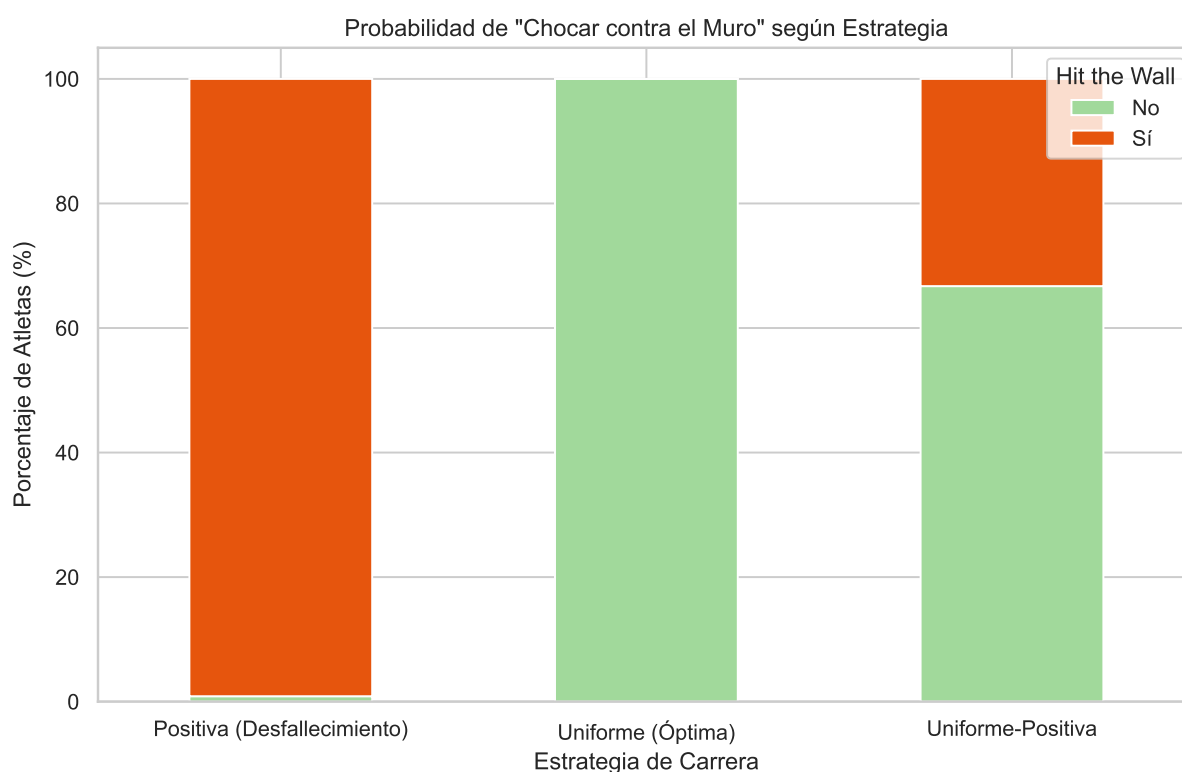
o el porcentaje de atletas (sobre el total de ese grupo) que hay en cada clúster.

Distribución de corredores por estrategia:

```
pacing_label
Uniforme (Óptima)          54.833118
Uniforme-Positiva         37.304841
Positiva (Desfallecimiento) 7.862041
Name: proportion, dtype: float64
```

Se observa que el 92% aproximadamente de los deportistas llevaron una estrategia muy estable, siendo la uniforme pura la más representada (54,8%).

Se procede ahora a evaluar la variable “Chocar contra el muro” con resultados objetivos como la estrategia y el tiempo final.



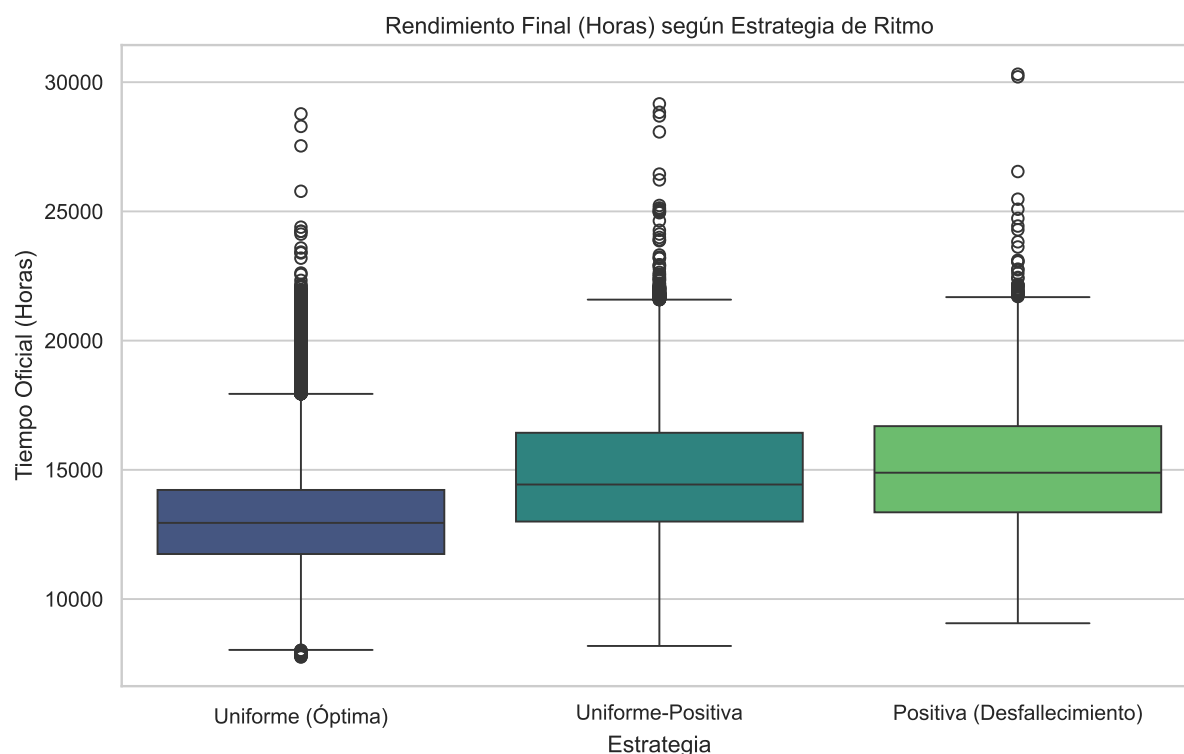
Este gráfico muestra la relación existente entre cada tipo de estrategia con el fenómeno de chocar contra el muro. Dicho fenómeno consiste en una bajada de rendimiento a partir de los segmentos centrales de la carrera (generalmente entre el km 20 y el km 30).

Se ve que las estrategias uniforme y positiva son completamente puras. Esto sugiere que aquellos atletas que adoptaron una estrategia positiva fue porque efectivamente chocaron contra el muro y tuvieron una bajada considerable de rendimiento a partir de uno de los tramos centrales de la maratón.

Aquellos que adoptaron una estrategia uniforme fueron capaces de dosificar sus fuerzas, lo que les permitió tolerar la fatiga y aguantar con un ritmo estable a lo largo del recorrido. No es que no acumularan fatiga, sino que gracias a un reparto eficiente del esfuerzo consiguieron minimizar los efectos de la acumulación de kilómetros.

En la estrategia Uniforme-Positiva se observa una división de los corredores. Más de un 60% no chocaron contra el muro, lo que podría traducirse en pequeñas fluctuaciones de ritmo durante toda la competición y no específicamente a partir de la mitad de la carrera.

El porcentaje restante si ha sido clasificado como que chocó contra el muro. Esto puede sugerir un ritmo estable hasta la mitad de la carrera y un ligero desfallecimiento desde un tramo central hasta el final. Estos atletas consiguieron aguantar la fatiga de la segunda mitad de la carrera y mantenerse relativamente estables, aunque a un ritmo inferior que el inicial.



En cuanto al tiempo final, se puede observar que cuanto más estable es la carrera, mejor tiempo se obtiene. Es decir, la eficiencia de una estrategia uniforme permite que el tiempo final acabe siendo más rápido. Esto comunica que en una prueba como la maratón no gana el que sale más rápido, sino el que aguanta el ritmo a lo largo de la carrera. No se trata de “ganar tiempo” en los primeros estadios, ya que esta ganancia se verá anulada al final por la fatiga. Al contrario, la mejor táctica consiste en ser capaz de minimizar los efectos de la fatiga, y en el mejor de los casos, incluso tener energía para acelerar en los últimos kilómetros.

Por último, se analiza la información que aportan los outliers a la dinámica de carrera de la maratón:

Cada boxplot representa una estrategia y la relaciona con el tiempo final en el eje Y.

En la estrategia uniforme los outliers situados encima del boxplot representan a todos aquellos corredores que llevaron una estrategia uniforme, pero a un ritmo significativamente más lento que la media. En el lado contrario, los outliers más rápidos son aquellos que llevaron una estrategia uniforme a una velocidad muy superior que el resto, estos son los atletas de élite.

En la estrategia positiva los outliers son los deportistas que no solo fueron más lento en términos absolutos, sino que además tuvieron una decaída en el rendimiento tan marcada que su tiempo final acabó siendo mayor. Esto da información acerca de las diferentes magnitudes de

desfallecimiento que se pueden dar dentro de una misma estrategia categorizada como positiva. Desde el decaimiento propio de la mayoría de corredores por la acumulación de fatiga hasta un decaimiento extremo, fruto de un comienzo demasiado rápido para las posibilidades físicas.

En la estrategia Uniforme-Positiva los outliers indican un comportamiento similar, además de ritmos absolutos más lentos, hubo diferentes magnitudes en las fluctuaciones sufridas a lo largo de los segmentos de la maratón.

Cruzar la variable “chocar contra el muro” con la estrategia y el tiempo final permite comprender cómo los procesos fisiológicos internos (chocar contra el muro implica una depleción en los depósitos musculares de glucógeno por la acumulación de kilómetros a una intensidad determinada) afectan a los resultados externos, objetivos y visibles: las estrategias y el tiempo final.

4.2 Clustering Jerárquico: Análisis Táctico de la Élite Femenina (2015)

Mientras que el algoritmo K-Means nos permitió identificar macro tendencias de ritmo en la población general de corredores, el **Clustering Jerárquico** es la herramienta ideal para disecionar grupos pequeños y específicos. En esta sección, abandonaremos el enfoque masivo para centrarnos en un ecosistema altamente competitivo: la categoría de Élite Femenina en la edición de 2015.

4.2.1 Fundamento Teórico: Enfoque Aglomerativo (*Bottom-Up*)

El algoritmo jerárquico aglomerativo asume inicialmente que cada observación (cada corredor) es un clúster independiente. A partir de ahí, el proceso iterativo es el siguiente:

1. **Cálculo de la Matriz de Distancias:** Se calcula la distancia entre todos los pares de clústeres existentes. La métrica más utilizada es la **Distancia Euclidiana** en un espacio de n dimensiones:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. **Fusión:** Se identifican los dos clústeres separados por la menor distancia y se unen para formar un nuevo clúster consolidado.
3. **Actualización:** Se recalcula la matriz de distancias para medir la lejanía entre el nuevo clúster formado y los clústeres restantes.
4. **Iteración:** Los pasos 2 y 3 se repiten hasta que todas las observaciones quedan englobadas en un único macro-clúster central.

4.2.1.1 Criterios de Enlace (*Linkage Criteria*)

El punto más crítico del algoritmo en el paso 3 es definir cómo se mide la distancia matemática entre dos grupos que contienen múltiples puntos. Para ello, existen diversos criterios de enlace. En nuestro análisis deportivo, evaluaremos principalmente:

- **Enlace Simple (*Single Linkage*):** Calcula la distancia entre los dos puntos más cercanos de cada clúster. Tiende a generar grupos alargados en forma de cadena.
- **Enlace Completo (*Complete Linkage*):** Calcula la distancia entre los dos puntos más alejados, favoreciendo la creación de grupos esféricos y compactos.

4.2.1.2 Representación Visual y Corte: El Dendrograma

El resultado de este proceso no es una simple asignación de etiquetas, sino una estructura de árbol denominada **Dendrograma**. En este gráfico, el eje vertical (altura) representa la distancia matemática a la que se han fusionado dos grupos.

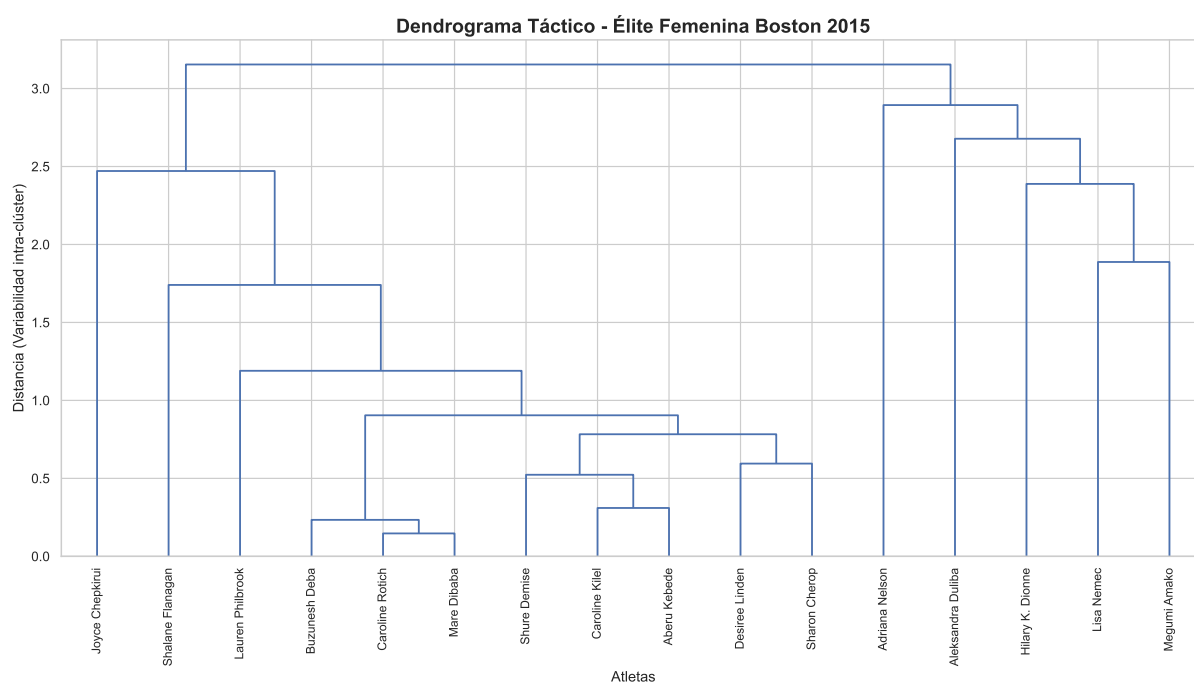
4.2.2 Análisis Táctico

Lo primero que se realizará será construir el dataframe con los datos de las atletas de Élite de la edición de 2015.

Total de corredoras en la Élite Femenina de 2015: 16

Una vez tenemos las variables, vamos a realizar y graficar el clustering jerárquico.

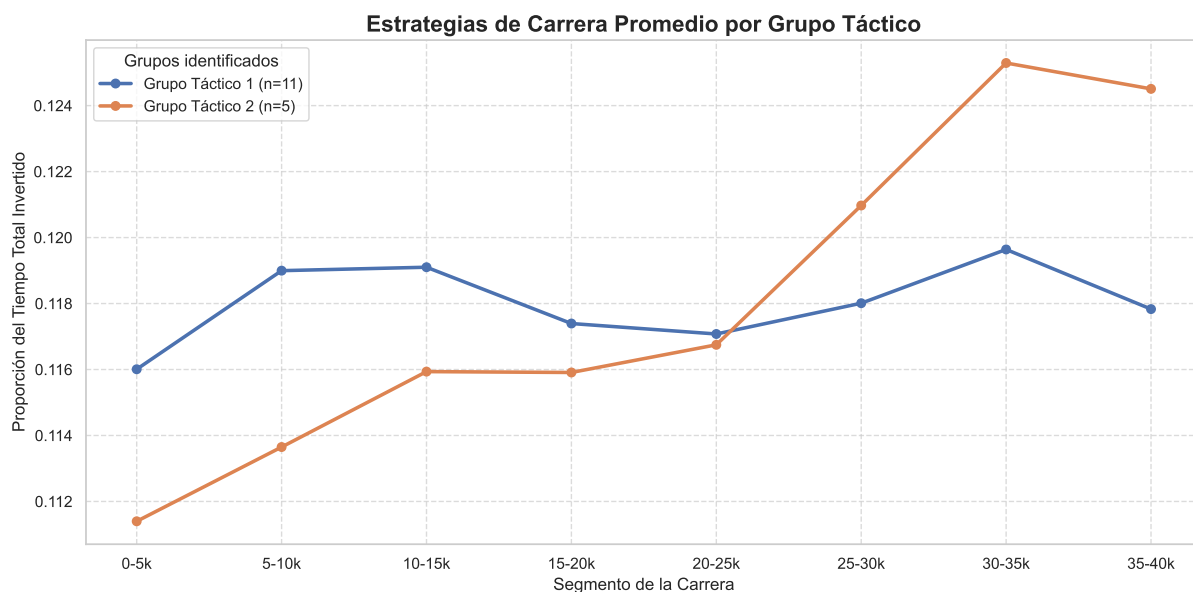
Comenzamos calculando el clustering jerárquico utilizando un enlace simple:



Tras visualizar el dendrograma, observamos que se pueden sacar 2 grupos distintos. Los dividimos y vemos sus centroides para ver las estrategias

Distribución de atletas por grupo táctico:

cluster_tactico	count
1	11
2	5



Podemos observar que el cluster 1 contiene 11 corredoras y es una estrategia mucho más constante que la del cluster 2, en la que es totalmente incremental.

El clustering jerárquico nos ha revelado la existencia de dos estrategias de carrera, pero... ¿cuál de ellas fue la más efectiva?

Para descubrirlo, cruzaremos las etiquetas de los grupos tácticos obtenidos con la posición final real de las atletas en su categoría (`division_result`). Calcularemos la posición media de cada pelotón e identificaremos en qué grupo se encontraba la ganadora absoluta de la prueba.

```
resumen_posiciones = df_elite_f_2015.groupby('cluster_tactico').agg(
    Num_Atletas=('division_result', 'count'),
    Posicion_Media=('division_result', 'mean'),
    Mejor_Posicion=('division_result', 'min')
).reset_index()

resumen_posiciones['Ganadora_en_grupo'] = resumen_posiciones['Mejor_Posicion'].apply(lambda

resumen_posiciones = resumen_posiciones.round({'Posicion_Media': 1})

resumen_posiciones.rename(columns={
    'cluster_tactico': 'Grupo Táctico',
    'Num_Atletas': 'Nº Atletas',
```

```

    'Posicion_Media': 'Posición Final Media',
    'Mejor_Posicion': 'Mejor Posición del Grupo',
    'Ganadora_en_grupo': '¿Contiene a la Ganadora?'
}, inplace=True)

print("\n")
print(resumen_posiciones.to_markdown(index=False))
print("\n")

```

Grupo Táctico	Nº Atletas	Posición Final Media	Mejor Posición del Grupo	¿Contiene a la Ganadora?
1	11	6.5	1	Sí
2	5	13	11	No

Por lo tanto observamos ¡que la estrategia constante (Cluster 1) es la mejor estrategia en cuanto a resultados obtenidos.

4.3 Clustering en Streaming: Simulación de Carrera en Vivo

Hasta ahora se ha visualizado la maratón como una foto fija de lo que ha ocurrido. A continuación, el enfoque cambia.

4.3.1 Fundamentación teórica

La premisa fundamental es que los datos no se tratan de forma global, sino que se procesan mediante micro-batches (lotes) conforme las atletas cruzan cada punto de control cronometrado (5k, 10k, 15k, etc.).

Este enfoque se basa en tres pilares operativos:

- **Ingesta por Ventanas Temporales:** El modelo opera bajo una restricción de “ceguera del futuro”. En cada etapa (por ejemplo, el km 20), el algoritmo de clustering solo tiene visibilidad de los datos acumulados hasta ese instante. Esto simula el flujo real de información en una retransmisión en vivo, permitiendo un análisis evolutivo y no retrospectivo.
- **Evolución Dinámica del Estado:** Al ejecutar el clustering de manera independiente en cada nuevo lote de datos, el sistema permite que el “estado” de una atleta sea fluido. Esto captura la historia del rendimiento: una corredora puede transicionar entre diferentes perfiles competitivos a lo largo de la prueba, permitiéndonos mapear su trayectoria completa en lugar de solo su resultado final.
- **Detección de Deriva de Datos (Drift):** El streaming permite identificar cambios de comportamiento en tiempo real mediante la comparación de los clusters entre batches consecutivos. Esta técnica es capaz de detectar la deriva de rendimiento (fatiga súbita o ataques tácticos) en el momento exacto en que ocurren. Es la diferencia entre saber cuánto tiempo ha perdido una atleta y entender dónde y por qué empezó a perderlo.

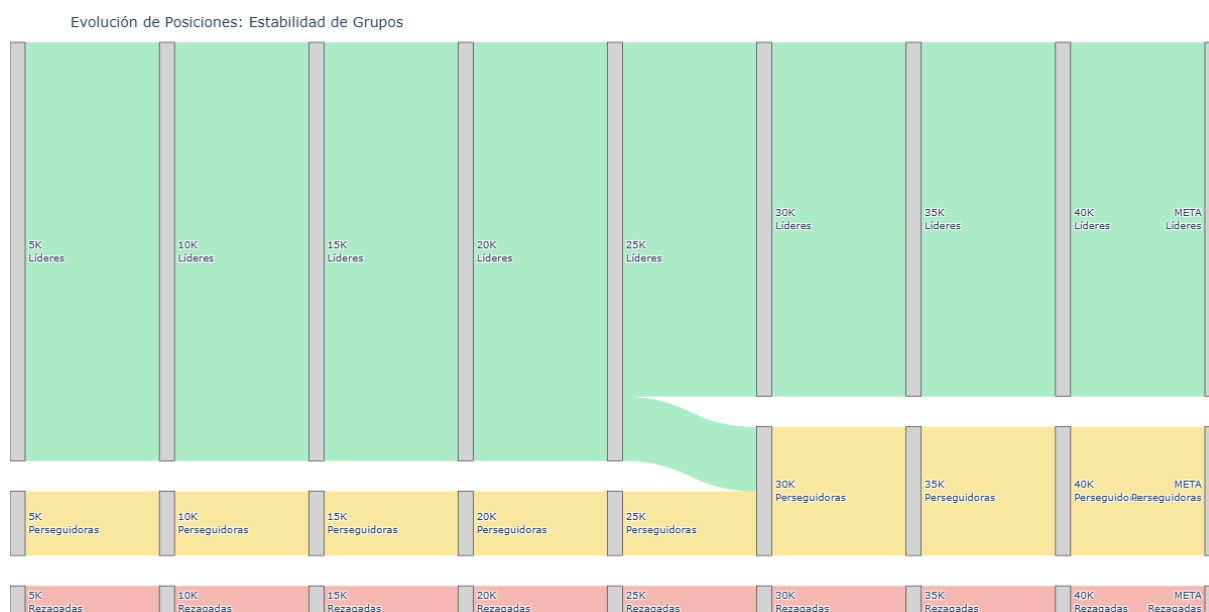
4.3.2 Clustering con grupos preestablecidos

El Clustering en streaming se hará primeramente pre-estableciendo grupos de carrera.

Se adoptarán dos enfoques distintos:

- **Enfoque Acumulado (Tiempos):** Captura la “posición relativa” total. Responde a: ¿En qué paquete de la carrera se encuentra la atleta?
- **Enfoque Puntual (Ritmos):** Captura el “esfuerzo actual”. Responde a: ¿Cómo está corriendo la atleta en este momento específico, independientemente de lo que hizo antes?

4.3.2.1 Enfoque Acumulado (Tiempos)

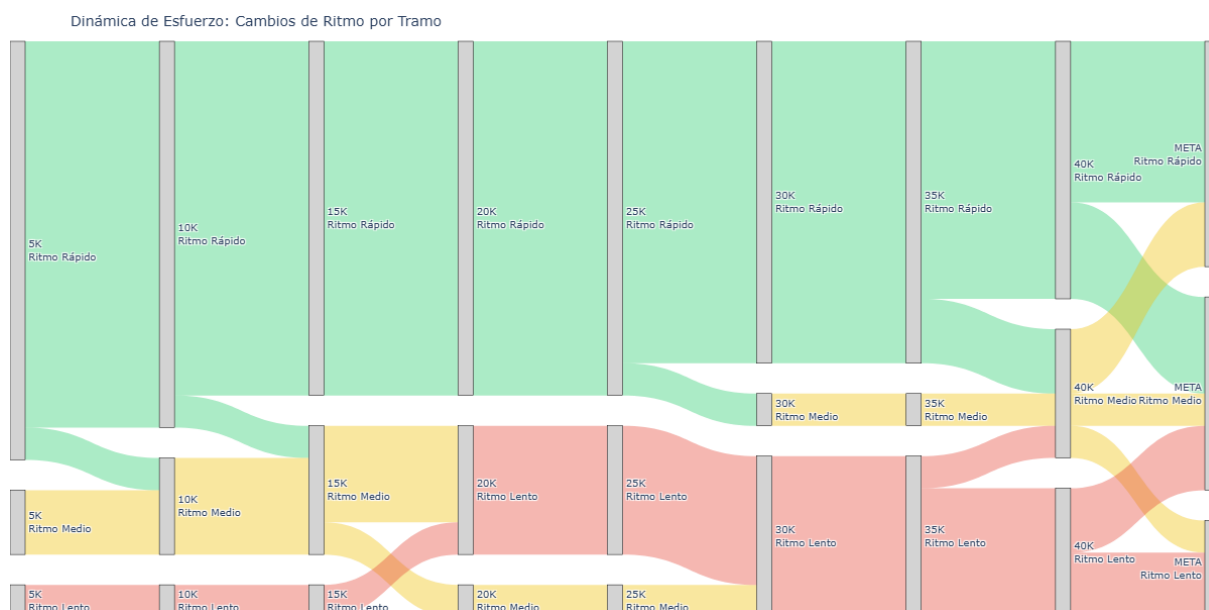


En este gráfico se pueden observar tres grupos bien diferenciados, siendo el que más participantes agrupa el de líderes. Solo se observa un cambio de grupo en el km 25, en el que un pequeño grupo de las líderes se descuelga y pasa al grupo de las perseguidoras.

Al haber preestablecido los grupos y aplicado el clustering sobre los tiempos apenas se han observado cambios. Esto ocurre porque a pesar de que pueda haber variaciones, el tiempo acumulado camufla los cambios. Por poner un ejemplo, si el grupo de cabeza se divide en 2 en el km 25 y el primer sub-grupo pasa por ese punto 20 segundos antes que el segundo sub-grupo, se seguirán clasificando como “Líderes”.

Al aplicar este clustering sobre el ritmo, se pueden captar algunos cambios más.

4.3.2.2 2. Enfoque acumulado: ritmos



Este gráfico muestra una cantidad de cambios considerablemente mayor al primero.

El grupo que lleva un ritmo rápido es bastante constante a lo largo de la carrera, solo con pequeñas escisiones de atletas que se ajustan a un ritmo más asequible, bien desde el principio o en momentos más centrales de la carrera.

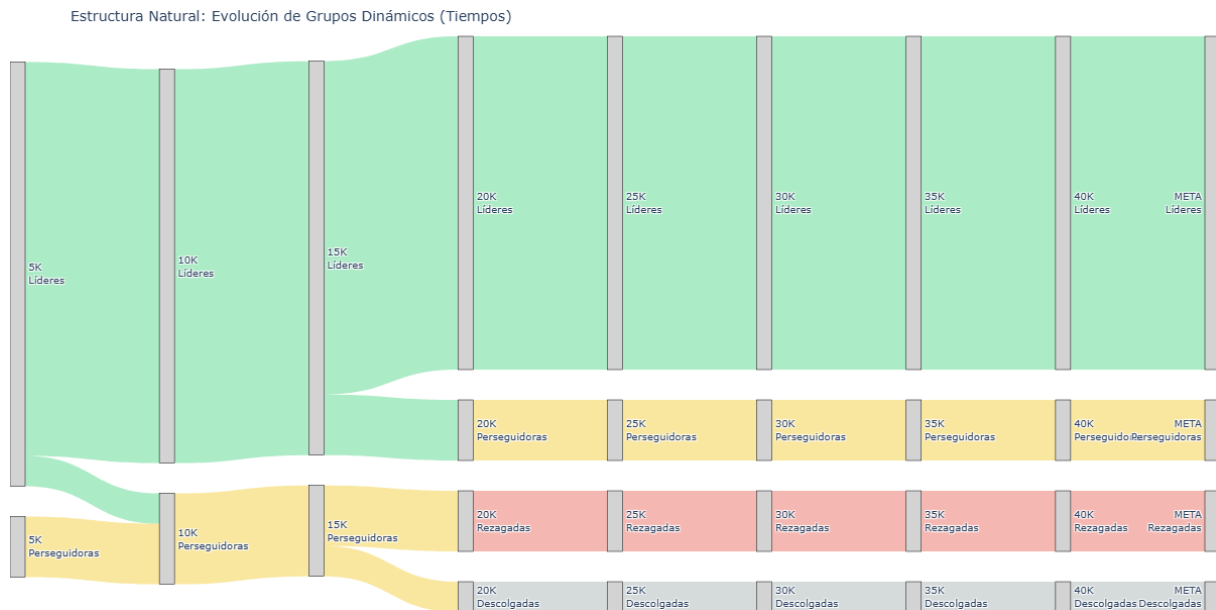
En cuanto al grupo que adopta un ritmo medio, se observa una disminución en su volumen a lo largo de la carrera. Por su parte, el grupo que lleva un ritmo lento se va haciendo cada vez más grande, acogiendo a los decaen del grupo medio y rápido.

Es de especial mención el tramo desde el km 40 hasta la meta, en el que se producen diversos cambios que dejan entrever las estrategias mencionadas anteriormente: algunos grupos se mantienen o aceleran en los últimos metros de la carrera mientras que otro, a pesar de la cercanía a la línea de llegada, no son capaces de mantener el ritmo y siguen decayendo hasta el final.

4.3.3 Clustering sin grupos preestablecidos

Con el objetivo de “dejar que los datos hablen” se procede a hacer un clustering dinámico sin indicar previamente ningún número de clusters. De esta forma, se podrán observar comportamientos más detallados que al forzar unos grupos como se ha hecho en el apartado anterior.

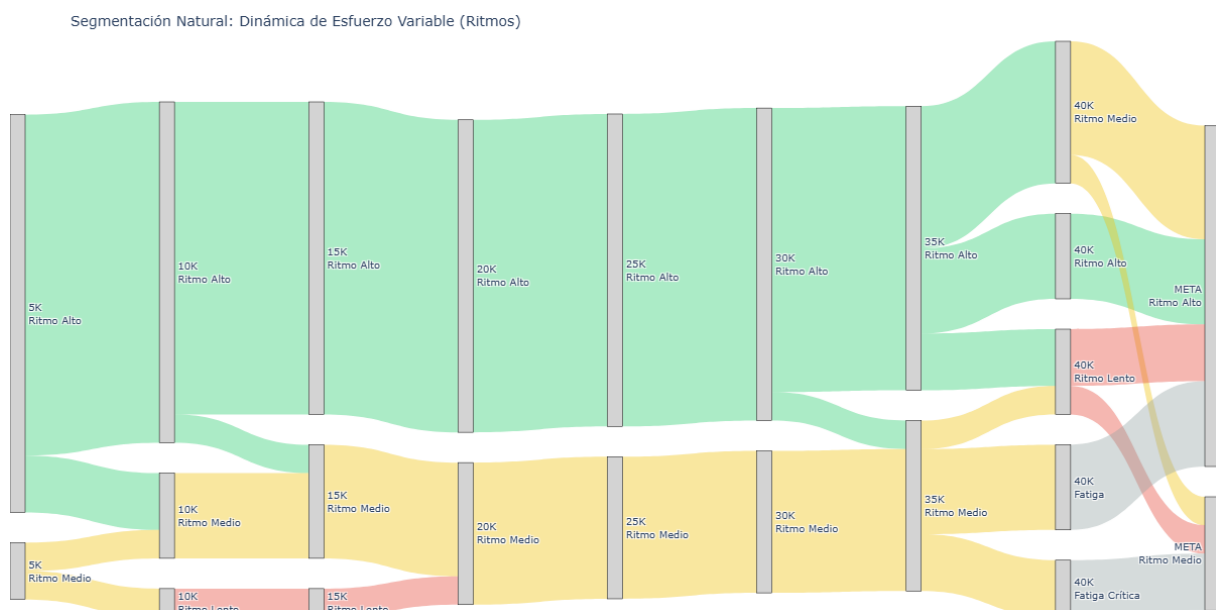
4.3.3.1 1. Enfoque acumulado: tiempos



Este gráfico muestra como al principio solo existían dos grupos que presentaban ligeras variaciones hasta el km 20. A partir de ese momento, la fatiga diferenció hasta grupos bien diferenciados y estables.

En comparación con los grupos pre-establecidos, llevar a cabo el clustering de forma más “libre” permite conocer con más detalle los grupos de carrera que se han formado.

4.3.3.2 2. Enfoque acumulado: ritmos



Resulta llamativo que al no preestablecer grupos, el algoritmo ha hecho una división inicial de dos ritmos. Esto sugiere que al obligar a encontrar 3 grupos se obligaba a ser mucho más sensible a cualquier diferencia de ritmo. En este caso, se es “más permisivo”, es decir, las pequeñas

variaciones de ritmo no se consideran como un grupo aparte, reservando esa clasificación para cuando la diferencia es más significativa.

En este mismo gráfico se observa como a lo largo de la carrera se diferencian mayoritariamente dos grupos relativamente estables pero en el tramo final detecta hasta 5 clasificaciones distintas, reflejando un gran cambio de ritmo en los 2 km y 195 m previos a la línea de meta.

En comparación con el gráfico de los tres grupos preestablecidos, esta figura prioriza los cambios importantes, independientemente de los grupos que se produzcan.

A modo de resumen, en este último gráfico se observa la diferencia entre el número de clusters detectado por el algoritmo en función de si se aplicaba sobre los tiempos o los ritmos. Esto confirma lo que ya se adelantaba anteriormente: los tiempos acumulados no reflejan los cambios de ritmo a nivel individual hasta que no son lo suficientemente llamativos como para provocar un cambio de grupo.

Unable to display output for mime type(s): text/html

Unable to display output for mime type(s): text/html

5 Aprendizaje Supervisado

Este capítulo se divide en dos problemas distintos:

1. **Problema de clasificación:** Predecir si los corredores “petan” o no en el muro a través de la variable creada en el análisis exploratorio.
2. **Problema de regresión:** Predecir el tiempo final de los atletas al paso por los 35km. Cabe destacar que en la asignatura de *Análisis de Datos en el Deporte* se realizó un estudio con modelos de regresión y se observó que el parcial que mejores predicciones obtenía era el 35, por lo tanto, ahora vamos a probar varios modelos sobre este parcial para determinar cuál es mejor.

5.1 Problema de clasificación. Detección de atletas que sufren en el muro.

Para comenzar esta sección, vamos a realizar la división de nuestro conjunto de datos en Train-Test-Validation.

Para ello, la mejor opción será realizar un muestreo estratificado ya que hay un 79.78% de corredores que no chocan con el muro `hit_the_wall=0`.

A continuación, se hace el muestreo estratificado en una proporción 60-20-20 para cada conjunto. Además, se dejan en X las variables predictoras que pueden llegar a usarse (age, gender, country_residence, 5k, 10k, 15k, 20k, half, temp_mean_c, humidity_pct, wind_speed_kmh).

```
features_todas = [
    'age', 'gender', 'country_residence',
    '5k', '10k', '15k', '20k', 'half',
    'temp_mean_c', 'humidity_pct', 'wind_speed_kmh'
]
target = 'hit_the_wall'

X = df_boston[features_todas]
y = df_boston[target]

# 3. DIVISIÓN ESTRATIFICADA (60/20/20)
# 60% Train / 40% Resto
X_train, X_temp, y_train, y_temp = train_test_split(
    X, y, test_size=0.40, random_state=42, stratify=y
)

# Mitad del resto para Validación (20%) y mitad para Test (20%)
```

```
X_val, X_test, y_val, y_test = train_test_split(
    X_temp, y_temp, test_size=0.50, random_state=42, stratify=y_temp
)
```

Una vez tenemos nuestros datos ya divididos, entrenaremos nuestros modelos en *Train*, los evaluaremos en *Test* y cuando tengamos todos los posibles modelos, elegiremos uno para probarlo en *Validation*.

Cabe destacar que evaluaremos el rendimiento de dichos modelos a través de las métricas:

- **Accuracy:** $Accuracy = \frac{TP+TN}{TP+TN+FN+FP}$
- **Recall:** Ya que decirle a un atleta que no va a golpearse contra el muro cuando en realidad sí es un error que puede costar muy caro, nuestro objetivo será maximizar dicha métrica, es decir, de todos los positivos reales, ¿cuántos caza el modelo? $Recall = \frac{TP}{TP+FN}$

Donde TP son los corredores que realmente chocan con el muro y el modelo detecta correctamente, y FN son los corredores que chocan con el muro pero el modelo no identifica. En este problema, reducir los FN es especialmente importante.

5.1.1 K-Vecinos Más Cercanos (K-NN)

El algoritmo de K-Vecinos Más Cercanos (K-NN) es un método de aprendizaje supervisado no paramétrico. A diferencia de otros algoritmos que construyen una ecuación matemática o un árbol de reglas durante el entrenamiento, K-NN es un algoritmo de tipo *lazy learning* (aprendizaje perezoso): no “aprende” un modelo en sí, sino que memoriza todo el conjunto de datos de entrenamiento para usarlo como mapa de referencia durante la predicción.

5.1.1.1 Fundamento del Algoritmo

Para clasificar a un nuevo corredor (es decir, predecir si sufrirá “El Muro” o no), el algoritmo sigue un proceso estrictamente geométrico:

1. **Cálculo de Distancias:** Mide la distancia matemática entre el nuevo corredor y todos los demás corredores presentes en el conjunto de entrenamiento. La métrica por defecto es la Distancia Euclidiana en un espacio n-dimensional:

$$d(x, y) = \sqrt{\sum_{i=1}^n (x_i - y_i)^2}$$

2. **Selección de la Vecindad (K):** Identifica a los K corredores del conjunto de entrenamiento que se encuentran a menor distancia del nuevo individuo. El valor K es un hiperparámetro crítico que se debe definir (generalmente un número impar para evitar empates, como 3, 5 o 7).
3. **Votación Mayoritaria:** El algoritmo observa la etiqueta real (`hit_the_wall`) de esos K vecinos más cercanos. La clase mayoritaria entre ellos será la predicción asignada al nuevo corredor.

5.1.1.2 Aplicación del Algoritmo

Una vez definido el marco teórico, aplicamos el algoritmo:

```
#1. Definimos las variables que vamos a usar:
variables_usar = ['age', '5k', '10k', '15k', '20k', 'half', 'temp_mean_c', 'humidity_pct', '
X_train_knn = X_train[variables_usar]
X_test_knn = X_test[variables_usar]
X_validation_knn = X_val[variables_usar]
#2. Las escalamos
scaler = StandardScaler()
X_train_knn = scaler.fit_transform(X_train_knn)
X_test_knn = scaler.transform(X_test_knn)
X_validation_knn = scaler.transform(X_validation_knn)
#3. Entrenamos el algoritmo
modelo_clasificacion_knn = KNeighborsClassifier(n_neighbors=1)
modelo_clasificacion_knn.fit(X_train_knn, y_train)
#4. Realizamos la predicción en test
predicciones_knn = modelo_clasificacion_knn.predict(X_test_knn)

#5. Visualizamos la matriz de confusión
tabla_confusion = pd.crosstab(
    y_test,
    predicciones_knn,
    rownames=['Realidad (0=No, 1=Sí)'],
    colnames=['Predicción (0=No, 1=Sí)']
)

acc_knn = accuracy_score(y_test, predicciones_knn)
rec_knn = recall_score(y_test, predicciones_knn, pos_label=1)

print("=== MATRIZ DE CONFUSIÓN (K-NN) ===\n")
print(tabla_confusion)

print("=== EVALUACIÓN DEL MODELO KNN ===")
print(f"Accuracy Global: {acc_knn:.2%}")
print(f"Recall (Detección del Muro): {rec_knn:.2%}")
```

```
=== MATRIZ DE CONFUSIÓN (K-NN) ===
```

Predicción (0=No, 1=Sí)	0	1
Realidad (0=No, 1=Sí)		
0	10655	1958
1	2124	1071

```
=== EVALUACIÓN DEL MODELO KNN ===
```

```
Accuracy Global: 74.18%
Recall (Detección del Muro): 33.52%
```

Comenzamos estableciendo un modelo base utilizando K-NN. Como era de esperar ante un

dataset desbalanceado (80/20), el algoritmo basado en distancias fue incapaz de identificar los colapsos, logrando un Recall máximo del 33% asumiendo un riesgo extremo de sobreajuste ($K = 1$). Esto justifica la necesidad de evolucionar hacia algoritmos basados en árboles que permitan un balanceo de pesos matemático.

5.1.2 Árbol de Decisión

El Árbol de Decisión (*Decision Tree*) es un algoritmo de aprendizaje supervisado no paramétrico que construye un modelo predictivo basado en una secuencia jerárquica de reglas lógicas. A diferencia de métodos basados en distancias (como K-NN) o en ecuaciones algebraicas (como la Regresión Lineal), el árbol divide el conjunto de datos iterativamente realizando preguntas binarias sobre las variables predictoras.

5.1.2.1 Funcionamiento y Criterio de Corte

En cada nodo, el algoritmo evalúa todas las características disponibles (edad, ritmos, país, etc.) y selecciona el punto de corte que mejor separa a los corredores según la variable objetivo (*hit_the_wall*). Para medir la calidad de esta separación, utiliza métricas matemáticas de “pureza”, siendo la más común el **Índice de Impureza de Gini**. El proceso se ramifica de forma descendente hasta alcanzar un criterio de parada definido por el analista (por ejemplo, una profundidad máxima, *max_depth*) para evitar el sobreajuste (*overfitting*).

5.1.2.2 Índice de Gini, Entropía y Error de Clasificación

Cuando un árbol prueba una posible división, no mira solo si acierta o falla en ese instante. Lo que intenta medir es si, después del corte, los nodos resultantes quedan más “puros”, es decir, con observaciones cada vez más concentradas en una única clase. Para ello se utilizan medidas de impureza.

Si en un nodo hay K clases y p_k representa la proporción de observaciones de la clase k , el índice de Gini se calcula como:

$$Gini = 1 - \sum_{k=1}^K p_k^2$$

Un nodo completamente puro tiene Gini igual a 0, porque todas las observaciones pertenecen a la misma clase. En cambio, si en un problema binario el nodo está dividido al 50%-50%, la impureza es alta, porque el árbol todavía no ha conseguido separar bien las clases.

La **entropía** mide una idea muy parecida, pero desde el punto de vista de la información o incertidumbre:

$$Entropia = - \sum_{k=1}^K p_k \log_2(p_k)$$

También vale 0 en nodos puros y aumenta cuando las clases están mezcladas. En la práctica, Gini y entropía suelen producir árboles bastante similares. Gini es algo más simple de calcular

y es el criterio por defecto en muchos árboles de clasificación; la entropía penaliza de forma algo más intensa ciertos nodos con mezcla de clases y se interpreta como ganancia de información.

Otra posibilidad sería usar el **error de clasificación**:

$$Error = 1 - \max(p_k)$$

Este indicador mira únicamente la clase mayoritaria del nodo. Por ejemplo, si un nodo tiene 60% de corredores que no chocan con el muro y 40% que sí chocan, su error sería 40%. El problema es que esta medida es menos sensible a pequeñas mejoras en la composición del nodo. Por eso suele ser menos útil para decidir los cortes durante el crecimiento del árbol, aunque puede ser intuitiva para explicar el error final de clasificación.

En este trabajo utilizamos `criterion='gini'` porque es un criterio robusto, eficiente y adecuado para construir árboles de clasificación. Además, combinado con `class_weight='balanced'`, permite que el árbol no ignore la clase minoritaria (`hit_the_wall=1`) durante el aprendizaje.

5.1.2.3 Aplicación del Algoritmo

A continuación, se realiza el algoritmo. La idea será coger las variables categóricas como el país y binarizarlas, en vez de dejar todos los posibles países, ya que son muchos. De esta manera, logramos una regla lógica de si el corredor es o no de los estados unidos. Tras ello, entrenamos el algoritmo:

```
X_train_tree = X_train.copy()
X_test_tree = X_test.copy()
X_val_tree = X_val.copy()
# Binarizamos la variable país
X_train_tree['is_USA'] = (X_train_tree['country_residence'] == 'USA').astype(int)
X_test_tree['is_USA'] = (X_test_tree['country_residence'] == 'USA').astype(int)
X_val_tree['is_USA'] = (X_val_tree['country_residence'] == 'USA').astype(int)

# Convertimos Género (1 = Mujer, 0 = Hombre)
X_train_tree['is_Female'] = (X_train_tree['gender'] == 'F').astype(int)
X_test_tree['is_Female'] = (X_test_tree['gender'] == 'F').astype(int)
X_val_tree['is_Female'] = (X_val_tree['gender'] == 'F').astype(int)
columnas_a_borrar = ['country_residence', 'gender']
X_train_tree.drop(columns=columnas_a_borrar, inplace=True)
X_test_tree.drop(columns=columnas_a_borrar, inplace=True)
X_val_tree.drop(columns=columnas_a_borrar, inplace=True)

modelo_clasificacion_arbol = DecisionTreeClassifier(
    criterion='gini',
    class_weight='balanced',
    max_depth=7,
    random_state=42
)
```



```

modelo_clasificacion_arbol.fit(X_train_tree, y_train)

predicciones_arbol = modelo_clasificacion_arbol.predict(X_test_tree)

tabla_arbol = pd.crosstab(
    y_test, predicciones_arbol,
    rownames=['Realidad (0=No, 1=Sí)'], colnames=['Predicción (0=No, 1=Sí)']
)

acc_arbol = accuracy_score(y_test, predicciones_arbol)
rec_arbol = recall_score(y_test, predicciones_arbol, pos_label=1)

print("=== MATRIZ DE CONFUSIÓN (ÁRBOL BALANCEADO) ===\n")
print(tabla_arbol)
print("\n=== EVALUACIÓN ===")
print(f"Accuracy Global: {acc_arbol:.2%}")
print(f"Recall (Detección del Muro): {rec_arbol:.2%}")

```

=== MATRIZ DE CONFUSIÓN (ÁRBOL BALANCEADO) ===

Predicción (0=No, 1=Sí)	0	1
Realidad (0=No, 1=Sí)		
0	7699	4914
1	888	2307

=== EVALUACIÓN ===

Accuracy Global:	63.30%
Recall (Detección del Muro):	72.21%

Este modelo, en comparación con el anterior, tiene mucha más *Recall* pero menos *Accuracy*.

La preparación de variables del árbol se ha mantenido sencilla e interpretable:

- **age**, los parciales hasta la media maratón (5k, 10k, 15k, 20k, half) y las variables meteorológicas se usan directamente como variables numéricas.
- **country_residence** se transforma en **is_USA**, una variable binaria que vale 1 si el corredor reside en Estados Unidos y 0 en caso contrario. Así evitamos introducir un número muy alto de categorías.
- **gender** se transforma en **is_Female**, que vale 1 para mujeres y 0 para hombres.
- No se escala ninguna variable porque los árboles no se basan en distancias, sino en cortes del tipo `variable <= umbral`.

En cuanto a los parámetros del modelo:

- **class_weight='balanced'** compensa el desbalanceo de **hit_the_wall**, dando más peso a la clase minoritaria.
- **criterion='gini'** define que los cortes se eligen minimizando la impureza de Gini.
- **max_depth=7** limita la profundidad del árbol. Es una forma de controlar el sobreajuste: si el árbol creciera sin límite podría aprender reglas demasiado específicas del conjunto de entrenamiento.

- `random_state=42` fija la semilla aleatoria para que los resultados sean reproducibles.

5.1.3 Random Forest

El algoritmo **Random Forest** es una extensión natural del Árbol de Decisión. En lugar de entrenar un único árbol, construye muchos árboles distintos y combina sus predicciones mediante votación mayoritaria. Cada árbol se entrena con una muestra aleatoria del conjunto de entrenamiento y, además, en cada corte solo considera una parte de las variables disponibles. Esta doble aleatoriedad hace que los árboles sean diferentes entre sí y reduce la varianza del modelo final.

La intuición es sencilla: un único árbol puede ser muy sensible a pequeñas variaciones en los datos, mientras que un bosque de árboles suele ser más estable. Para este problema mantenemos el mismo preprocesamiento usado en el árbol individual, ya que Random Forest también trabaja bien con variables no escaladas y con variables binarias creadas a partir de categorías.

5.1.3.1 Aplicación del Algoritmo

```
modelo_clasificacion_rf = RandomForestClassifier(
    n_estimators=300,
    criterion='gini',
    class_weight='balanced_subsample',
    max_depth=7,
    max_features='sqrt',
    min_samples_leaf=20,
    random_state=42,
    n_jobs=-1
)

modelo_clasificacion_rf.fit(X_train_tree, y_train)

predicciones_rf = modelo_clasificacion_rf.predict(X_test_tree)

tabla_rf = pd.crosstab(
    y_test, predicciones_rf,
    rownames=['Realidad (0=No, 1=Sí)'],
    colnames=['Predicción (0=No, 1=Sí)']
)

acc_rf = accuracy_score(y_test, predicciones_rf)
rec_rf = recall_score(y_test, predicciones_rf, pos_label=1)

importancias_rf = (
    pd.DataFrame({
        'Variable': X_train_tree.columns,
        'Importancia': modelo_clasificacion_rf.feature_importances_
    })
)
```

```

        .sort_values('Importancia', ascending=False)
        .reset_index(drop=True)
    )

print("=== MATRIZ DE CONFUSIÓN (RANDOM FOREST BALANCEADO) ===\n")
print(tabla_rf)
print("\n=== EVALUACIÓN ===")
print(f"Accuracy Global: {acc_rf:.2%}")
print(f"Recall (Detección del Muro): {rec_rf:.2%}")
print("\n=== VARIABLES MÁS IMPORTANTES ===")
display(importancias_rf.head(10))

```

=== MATRIZ DE CONFUSIÓN (RANDOM FOREST BALANCEADO) ===

Predicción (0=No, 1=Sí)	0	1
Realidad (0=No, 1=Sí)		
0	7571	5042
1	784	2411

=== EVALUACIÓN ===

Accuracy Global:	63.15%
Recall (Detección del Muro):	75.46%

=== VARIABLES MÁS IMPORTANTES ===

	Variable	Importancia
0	is_Female	0.212594
1	half	0.132783
2	humidity_pct	0.110042
3	temp_mean_c	0.102492
4	20k	0.098802
5	wind_speed_kmh	0.095339
6	10k	0.082027
7	15k	0.070149
8	5k	0.064027
9	age	0.029158

Los parámetros elegidos tienen la siguiente función:

- `n_estimators=300` indica que el bosque estará formado por 300 árboles. Aumentar este valor suele estabilizar el resultado, aunque incrementa el tiempo de entrenamiento.
- `criterion='gini'` mantiene el mismo criterio de pureza usado en el árbol individual.
- `class_weight='balanced_subsample'` corrige el desbalanceo de clases dentro de cada muestra aleatoria usada para entrenar cada árbol.
- `max_depth=7` mantiene árboles de complejidad comparable al árbol individual anterior.
- `max_features='sqrt'` hace que cada árbol valore solo una raíz cuadrada del número total de variables en cada división, aumentando la diversidad del bosque.

- `min_samples_leaf=20` exige que cada hoja final tenga al menos 20 observaciones, suavizando reglas demasiado específicas.
- `n_jobs=-1` permite usar todos los núcleos disponibles para acelerar el entrenamiento.

5.1.4 Selección del mejor modelo y predicción en el conjunto de Validación

Tras haber entrenado y probado en test los 3 modelos anteriores, comparamos sus resultados usando como criterio principal el *Recall*. En este problema nos interesa detectar el mayor número posible de corredores que realmente chocan con el muro; por eso, si dos modelos tienen un rendimiento similar, priorizamos el que capture mejor la clase positiva. La *Accuracy* se utiliza como criterio secundario para evitar elegir un modelo que detecte muchos casos positivos a costa de fallar en exceso sobre el conjunto global.

En esta comparación final no usamos Gini ni entropía porque no son métricas externas de rendimiento, sino criterios internos de aprendizaje propios de los árboles. K-NN, por ejemplo, no genera nodos ni cortes, por lo que no tendría sentido compararlo con Random Forest usando impureza de Gini. Para comparar modelos diferentes necesitamos una medida común basada en sus predicciones sobre los mismos datos de `test`. Por eso usamos la matriz de confusión, la *Accuracy*, el *Recall* y, de forma implícita, los errores que cada modelo comete sobre corredores que chocan o no chocan con el muro.

También incorporamos la curva **ROC** y el **AUC-ROC**. Esta técnica es propia de clasificación binaria y evalúa la capacidad del modelo para ordenar correctamente a los corredores según su probabilidad de chocar con el muro. La curva ROC compara la tasa de verdaderos positivos frente a la tasa de falsos positivos para distintos umbrales de decisión. El AUC resume esa curva: cuanto más cercano a 1, mejor separa el modelo ambas clases.

Por ello, se realiza primero una tabla comparativa en `test` y después se evalúa en `validation` únicamente el modelo seleccionado:

```
prob_knn_test = modelo_clasificacion_knn.predict_proba(X_test_knn)[: , 1]
prob_arbol_test = modelo_clasificacion_arbol.predict_proba(X_test_tree)[: , 1]
prob_rf_test = modelo_clasificacion_rf.predict_proba(X_test_tree)[: , 1]

metricas_test = pd.DataFrame({
    'Modelo': ['K-NN', 'Árbol de Decisión', 'Random Forest'],
    'Accuracy': [acc_knn, acc_arbol, acc_rf],
    'Recall': [rec_knn, rec_arbol, rec_rf],
    'AUC_ROC': [
        roc_auc_score(y_test, prob_knn_test),
        roc_auc_score(y_test, prob_arbol_test),
        roc_auc_score(y_test, prob_rf_test)
    ]
})

metricas_test = (
    metricas_test
    .sort_values(['Recall', 'Accuracy'], ascending=False)
    .reset_index(drop=True)
)
```

```

metricas_test_mostrar = metricas_test.copy()
metricas_test_mostrar['Accuracy'] = metricas_test_mostrar['Accuracy'].map(lambda x: f"{x:.2%}")
metricas_test_mostrar['Recall'] = metricas_test_mostrar['Recall'].map(lambda x: f"{x:.2%}")
metricas_test_mostrar['AUC_ROC'] = metricas_test_mostrar['AUC_ROC'].map(lambda x: f"{x:.3f}")

print("=== COMPARACIÓN DE MODELOS EN TEST ===")
display(metricas_test_mostrar)

plt.figure(figsize=(8, 6))

for nombre_modelo, probabilidades in {
    'K-NN': prob_knn_test,
    'Árbol de Decisión': prob_arbol_test,
    'Random Forest': prob_rf_test
}.items():
    fpr, tpr, _ = roc_curve(y_test, probabilidades)
    auc_modelo = roc_auc_score(y_test, probabilidades)
    plt.plot(fpr, tpr, label=f'{nombre_modelo} (AUC={auc_modelo:.3f})')

plt.plot([0, 1], [0, 1], linestyle='--', color='gray', label='Azar (AUC=0.500)')
plt.xlabel('Tasa de falsos positivos')
plt.ylabel('Tasa de verdaderos positivos')
plt.title('Curvas ROC en el conjunto de test')
plt.legend()
plt.show()

mejor_modelo = metricas_test.loc[0, 'Modelo']

modelos_validacion = {
    'K-NN': (modelo_clasificacion_knn, X_validation_knn),
    'Árbol de Decisión': (modelo_clasificacion_arbol, X_val_tree),
    'Random Forest': (modelo_clasificacion_rf, X_val_tree)
}

modelo_final, X_validacion_final = modelos_validacion[mejor_modelo]

predicciones_validacion = modelo_final.predict(X_validacion_final)

tabla_validacion = pd.crosstab(
    y_val, predicciones_validacion,
    rownames=['Realidad (0=No, 1=Sí)'], colnames=['Predicción (0=No, 1=Sí)']
)

acc_validacion = accuracy_score(y_val, predicciones_validacion)
rec_validacion = recall_score(y_val, predicciones_validacion, pos_label=1)

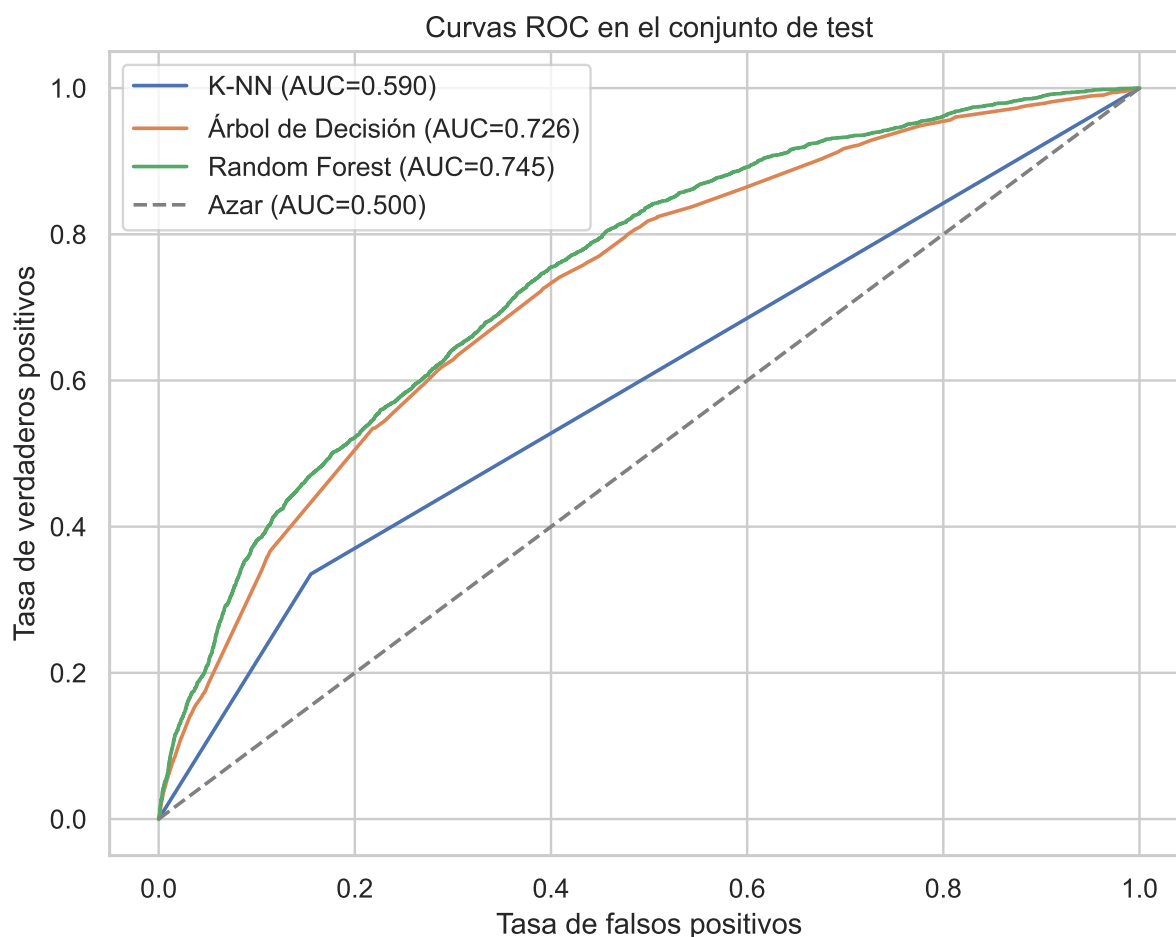
print(f"\nModelo seleccionado: {mejor_modelo}")
print("\n=== MATRIZ DE CONFUSIÓN EN VALIDACIÓN ===\n")

```

```
print(tabla_validacion)
print("\n=== EVALUACIÓN EN VALIDACIÓN ===")
print(f"Accuracy Global:          {acc_validacion:.2%}")
print(f"Recall (Detección del Muro): {rec_validacion:.2%}")
```

=== COMPARACIÓN DE MODELOS EN TEST ===

	Modelo	Accuracy	Recall	AUC_ROC
0	Random Forest	63.15%	75.46%	0.745
1	Árbol de Decisión	63.30%	72.21%	0.726
2	K-NN	74.18%	33.52%	0.590



Modelo seleccionado: Random Forest

=== MATRIZ DE CONFUSIÓN EN VALIDACIÓN ===

```
Predicción (0=No, 1=Sí)    0    1
Realidad (0=No, 1=Sí)
```

0	7554	5058
1	816	2380

=== EVALUACIÓN EN VALIDACIÓN ===

Accuracy Global: 62.84%

Recall (Detección del Muro): 74.47%

Con esta comparación, K-NN queda como modelo base: tiene una *Accuracy* razonable, pero su bajo *Recall* lo hace poco adecuado para detectar a los corredores que sí sufren en el muro. El Árbol de Decisión mejora claramente la detección de la clase positiva gracias al balanceo de clases, aunque sacrifica precisión global. Random Forest busca un punto más estable entre ambos enfoques: conserva la capacidad de los árboles para capturar relaciones no lineales y reduce la dependencia de una única estructura de decisión.

5.2 Problema de regresión. Predicción de tiempo final según sus datos hasta el parcial 35km

El objetivo de esta sección es predecir el tiempo final oficial de los corredores (*official_time*) utilizando únicamente la información disponible hasta el paso por el kilómetro 35. Es decir, el modelo no puede usar variables posteriores como 40k, *pace*, posiciones finales o cualquier otra variable que dependa de haber terminado la carrera, ya que eso supondría introducir fuga de información (*data leakage*).

En la asignatura de *Análisis de Datos en el Deporte* se observó que el parcial de 35 km era especialmente informativo para estimar el tiempo final. En este capítulo lo vamos a tratar como un problema de regresión supervisada y probaremos varios **Modelos Lineales Generalizados (GLM)** para determinar cuál funciona mejor.

Aunque en sentido estadístico estricto un GLM puede incluir distintas distribuciones y funciones de enlace, en este análisis trabajaremos con modelos lineales de regresión implementados en **scikit-learn**: Regresión Lineal, Ridge y Lasso. Los tres comparten una idea central: estimar el tiempo final como una combinación lineal de variables predictoras.

5.2.1 Planteamiento del problema y selección de variables

La variable objetivo será:

- **official_time**: tiempo final oficial en segundos.

Las variables predictoras serán aquellas que un analista podría conocer en el kilómetro 35:

- Variables sociodemográficas: *age*, *gender*, *country_residence*.
- Parciales acumulados hasta el 35 km: 5k, 10k, 15k, 20k, *half*, 25k, 30k, 35k.
- Condiciones meteorológicas: *temp_mean_c*, *humidity_pct*, *wind_speed_kmh*.

Dejamos fuera 40k, *pace*, *projected_time*, clasificaciones finales y *hit_the_wall*, porque contienen información posterior o derivada del resultado final.

Para validar los modelos mantendremos la misma filosofía que en clasificación:

- **Train (60%)**: datos usados para ajustar los coeficientes de cada modelo.
- **Test (20%)**: datos usados para comparar modelos y elegir hiperparámetros sencillos, como `alpha` en Ridge y Lasso.
- **Validation (20%)**: datos reservados para una evaluación final del modelo seleccionado.

En regresión no usamos muestreo estratificado porque la variable objetivo es continua. Antes de dividir, eliminamos cualquier fila con valores nulos en las variables usadas para que todos los modelos trabajen sobre exactamente el mismo conjunto de observaciones.

```
features_regresion = [
    'age', 'gender', 'country_residence',
    '5k', '10k', '15k', '20k', 'half', '25k', '30k', '35k',
    'temp_mean_c', 'humidity_pct', 'wind_speed_kmh'
]

target_regresion = 'official_time'

df_regresion = df_boston[features_regresion + [target_regresion]].dropna().copy()

X_reg = df_regresion[features_regresion].copy()
y_reg = df_regresion[target_regresion].copy()

X_train_reg, X_temp_reg, y_train_reg, y_temp_reg = train_test_split(
    X_reg, y_reg, test_size=0.40, random_state=42
)

X_val_reg, X_test_reg, y_val_reg, y_test_reg = train_test_split(
    X_temp_reg, y_temp_reg, test_size=0.50, random_state=42
)
```

5.2.2 Distribución de la variable objetivo

Antes de entrenar modelos lineales conviene estudiar la distribución de `official_time`. Esto no significa que la Regresión Lineal exija que la variable objetivo sea perfectamente normal. La hipótesis clásica de normalidad se refiere sobre todo a los **residuos** del modelo, no directamente a `y`. Aun así, mirar la distribución del tiempo final es útil por tres motivos:

- Permite detectar asimetrías fuertes o valores extremos.
- Ayuda a interpretar si métricas como MAE y RMSE serán sensibles a corredores muy lentos o muy rápidos.
- Permite comprobar si el problema tiene una escala razonablemente continua para abordarlo como regresión.

```
tiempo_final_min = df_regresion[target_regresion] / 60

resumen_tiempo_final = pd.DataFrame({
    'Métrica': ['Media', 'Mediana', 'Desviación típica', 'Mínimo', 'Máximo', 'Asimetría'],
    'Valor': [
        tiempo_final_min.mean(),
```



```

    tiempo_final_min.median(),
    tiempo_final_min.std(),
    tiempo_final_min.min(),
    tiempo_final_min.max(),
    stats.skew(tiempo_final_min)
]
})

display(resumen_tiempo_final)

fig, axes = plt.subplots(1, 2, figsize=(14, 5))

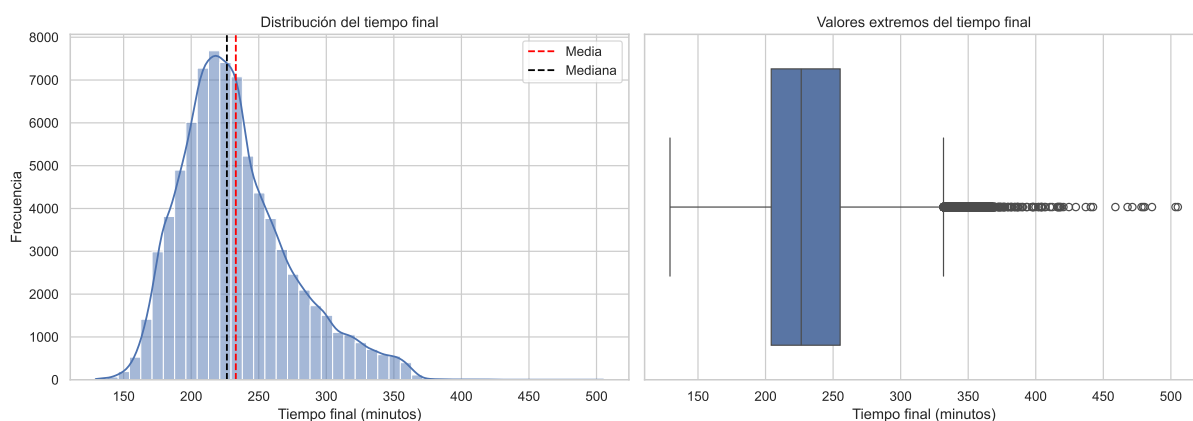
sns.histplot(tiempo_final_min, bins=45, kde=True, ax=axes[0])
axes[0].axvline(tiempo_final_min.mean(), color='red', linestyle='--', label='Media')
axes[0].axvline(tiempo_final_min.median(), color='black', linestyle='--', label='Mediana')
axes[0].set_title('Distribución del tiempo final')
axes[0].set_xlabel('Tiempo final (minutos)')
axes[0].set_ylabel('Frecuencia')
axes[0].legend()

sns.boxplot(x=tiempo_final_min, ax=axes[1])
axes[1].set_title('Valores extremos del tiempo final')
axes[1].set_xlabel('Tiempo final (minutos)')

plt.tight_layout()
plt.show()

```

	Métrica	Valor
0	Media	233.028009
1	Mediana	226.416667
2	Desviación típica	41.445240
3	Mínimo	129.283333
4	Máximo	505.150000
5	Asimetría	0.823105



Si la distribución presenta cierta asimetría hacia tiempos altos, no invalida automáticamente el uso de GLM. Lo importante será comprobar después si los errores del modelo se comportan de forma razonable: que no haya patrones claros en los residuos, que los errores extremos no dominen el RMSE y que el rendimiento se mantenga estable en `validation`.

5.2.3 Preprocesamiento para modelos lineales

Los modelos lineales necesitan que todas las variables sean numéricas. Por eso transformamos las variables categóricas en variables binarias:

- `is_Female`: vale 1 si el corredor es mujer y 0 si es hombre.
- `is_USA`: vale 1 si el corredor reside en Estados Unidos y 0 en caso contrario.

Además, escalamos las variables con `StandardScaler`. Este paso no cambia la información contenida en los datos, pero sí coloca todas las variables en una escala comparable. Es especialmente importante en Ridge y Lasso porque ambos modelos penalizan el tamaño de los coeficientes; si una variable está medida en una escala mucho mayor que otra, la penalización podría ser injusta.

```
def preparar_datos_regresion(X):
    X_preparado = X.copy()
    X_preparado['is_Female'] = (X_preparado['gender'] == 'F').astype(int)
    X_preparado['is_USA'] = (X_preparado['country_residence'] == 'USA').astype(int)
    X_preparado.drop(columns=['gender', 'country_residence'], inplace=True)
    return X_preparado

X_train_reg_prep = preparar_datos_regresion(X_train_reg)
X_test_reg_prep = preparar_datos_regresion(X_test_reg)
X_val_reg_prep = preparar_datos_regresion(X_val_reg)

scaler_reg = StandardScaler()
X_train_reg_scaled = scaler_reg.fit_transform(X_train_reg_prep)
X_test_reg_scaled = scaler_reg.transform(X_test_reg_prep)
X_val_reg_scaled = scaler_reg.transform(X_val_reg_prep)
```

5.2.4 Métricas de evaluación en regresión

En clasificación evaluábamos si el modelo acertaba o fallaba una clase. En regresión, en cambio, el modelo predice un número continuo: el tiempo final en segundos. Por eso necesitamos métricas que midan la distancia entre el tiempo real y el tiempo predicho.

Utilizaremos tres métricas principales:

- **MAE (*Mean Absolute Error*)**: error absoluto medio. Indica, en segundos, cuánto se equivoca el modelo de media.

$$MAE = \frac{1}{n} \sum_{i=1}^n |y_i - \hat{y}_i|$$

- **RMSE (*Root Mean Squared Error*)**: raíz del error cuadrático medio. También se expresa en segundos, pero penaliza más los errores grandes.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (y_i - \hat{y}_i)^2}$$

- R^2 : proporción de variabilidad del tiempo final explicada por el modelo. Cuanto más cercano a 1, mejor ajuste.

$$R^2 = 1 - \frac{\sum_{i=1}^n (y_i - \hat{y}_i)^2}{\sum_{i=1}^n (y_i - \bar{y})^2}$$

Para elegir el mejor modelo usaremos principalmente el **MAE**, porque es fácil de interpretar deportivamente: si el MAE es 120, significa que el modelo se equivoca de media en unos 2 minutos. El RMSE será una métrica secundaria para controlar si algún modelo comete errores grandes, y R^2 nos servirá para interpretar la capacidad explicativa global.

Las curvas ROC no se aplican directamente a esta parte porque ROC evalúa clasificadores binarios, no modelos que predicen un valor continuo como segundos de carrera. Si quisiéramos usar ROC en este contexto tendríamos que transformar el problema en clasificación, por ejemplo predecir si un corredor bajará de 3 horas o no. Como aquí queremos predecir el tiempo exacto, las herramientas equivalentes son el análisis de residuos, los gráficos real vs predicho y las métricas de error.

```
def segundos_a_min_seg(segundos):
    segundos = int(round(segundos))
    minutos = segundos // 60
    segundos_restantes = segundos % 60
    return f"{minutos} min {segundos_restantes} s"

def evaluar_modelo_regresion(nombre_modelo, modelo, X_test, y_test):
    predicciones = modelo.predict(X_test)
    mae = mean_absolute_error(y_test, predicciones)
    rmse = np.sqrt(mean_squared_error(y_test, predicciones))
    r2 = r2_score(y_test, predicciones)

    return {
        'Modelo': nombre_modelo,
        'MAE': mae,
        'RMSE': rmse,
        'R2': r2,
        'MAE_interpretable': segundos_a_min_seg(mae),
        'RMSE_interpretable': segundos_a_min_seg(rmse)
    }
```

5.2.5 GLM 1: Regresión Lineal

La **Regresión Lineal** es el modelo base. Busca estimar el tiempo final como una suma ponderada de las variables:

$$\hat{y} = \beta_0 + \beta_1 x_1 + \beta_2 x_2 + \dots + \beta_p x_p$$

Cada coeficiente β_j indica cuánto cambia la predicción cuando aumenta una variable, manteniendo constantes las demás. En este caso, como las variables están escaladas, los coeficientes permiten comparar qué variables tienen mayor peso relativo dentro del modelo.

La ventaja de este modelo es su interpretabilidad. La limitación es que no regulariza los coeficientes: si hay variables muy correlacionadas entre sí, como los parciales acumulados de carrera, el modelo puede volverse inestable.

```

modelo_regresion_lineal = LinearRegression()
modelo_regresion_lineal.fit(X_train_reg_scaled, y_train_reg)

resultado_lineal = evaluar_modelo_regresion(
    'Regresión Lineal',
    modelo_regresion_lineal,
    X_test_reg_scaled,
    y_test_reg
)

print("=== EVALUACIÓN DEL MODELO: REGRESIÓN LINEAL ===")
print(f"MAE:    {resultado_lineal['MAE']:.2f} segundos ({resultado_lineal['MAE_interpretable']:.2f} segundos)")
print(f"RMSE:   {resultado_lineal['RMSE']:.2f} segundos ({resultado_lineal['RMSE_interpretable']:.2f} segundos)")
print(f"R2:     {resultado_lineal['R2']:.4f}")

```

```

=== EVALUACIÓN DEL MODELO: REGRESIÓN LINEAL ===
MAE:    140.39 segundos (2 min 20 s)
RMSE:   253.86 segundos (4 min 14 s)
R2:     0.9894

```

5.2.6 GLM 2: Ridge Regression

La **Regresión Ridge** parte de la Regresión Lineal, pero añade una penalización sobre el tamaño de los coeficientes. Esta penalización se conoce como regularización L2:

$$L2 = \alpha \sum_{j=1}^p \beta_j^2$$

El parámetro α controla la fuerza de la penalización:

- Si α es pequeño, Ridge se parece mucho a una Regresión Lineal.
- Si α es grande, los coeficientes se reducen más y el modelo se vuelve más conservador.

Ridge es especialmente útil cuando las variables están correlacionadas. En nuestro caso, los parciales 25k, 30k y 35k contienen información muy relacionada, porque todos son tiempos acumulados de carrera. Ridge ayuda a repartir el peso entre esas variables sin que un coeficiente se dispare de forma artificial.

Probamos varios valores de `alpha` usando el conjunto de `test` como referencia de comparación inicial:

```

alphas_ridge = [0.01, 0.1, 1, 10, 100]
resultados_ridge = []
modelos_ridge = {}

for alpha in alphas_ridge:
    modelo_ridge = Ridge(alpha=alpha)
    modelo_ridge.fit(X_train_reg_scaled, y_train_reg)
    modelos_ridge[alpha] = modelo_ridge

    resultado = evaluar_modelo_regresion(
        f'Ridge alpha={alpha}',
        modelo_ridge,
        X_test_reg_scaled,
        y_test_reg
    )
    resultado['Alpha'] = alpha
    resultados_ridge.append(resultado)

tabla_ridge = pd.DataFrame(resultados_ridge).sort_values('MAE').reset_index(drop=True)
display(tabla_ridge[['Modelo', 'MAE', 'RMSE', 'R2', 'MAE_interpretable', 'RMSE_interpretable']])

mejor_alpha_ridge = tabla_ridge.loc[0, 'Alpha']
modelo_ridge_final = modelos_ridge[mejor_alpha_ridge]

print(f"Mejor alpha para Ridge según MAE en test: {mejor_alpha_ridge}")

```

	Modelo	MAE	RMSE	R2	MAE_interpretable	RMSE_interpretable
0	Ridge alpha=0.01	140.395199	253.857234	0.989408	2 min 20 s	4 min 14 s
1	Ridge alpha=0.1	140.413002	253.852720	0.989408	2 min 20 s	4 min 14 s
2	Ridge alpha=1	140.591434	253.813258	0.989411	2 min 21 s	4 min 14 s
3	Ridge alpha=10	142.395241	253.818332	0.989411	2 min 22 s	4 min 14 s
4	Ridge alpha=100	157.240948	264.872117	0.988468	2 min 37 s	4 min 25 s

Mejor alpha para Ridge según MAE en test: 0.01

5.2.7 GLM 3: Lasso Regression

La **Regresión Lasso** también regulariza la Regresión Lineal, pero utiliza penalización L1:

$$L1 = \alpha \sum_{j=1}^p |\beta_j|$$

La diferencia clave es que Lasso puede llevar algunos coeficientes exactamente a 0. Esto significa que, además de ajustar el modelo, puede actuar como una técnica de selección automática de variables.

En este problema, Lasso nos ayuda a comprobar si todas las variables disponibles hasta el kilómetro 35 aportan información o si algunas son redundantes. Igual que con Ridge, el parámetro `alpha` controla la intensidad de la penalización.

```
alphas_lasso = [0.01, 0.1, 1, 10, 100]
resultados_lasso = []
modelos_lasso = {}

for alpha in alphas_lasso:
    modelo_lasso = Lasso(alpha=alpha, max_iter=10000)
    modelo_lasso.fit(X_train_reg_scaled, y_train_reg)
    modelos_lasso[alpha] = modelo_lasso

    resultado = evaluar_modelo_regresion(
        f'Lasso alpha={alpha}',
        modelo_lasso,
        X_test_reg_scaled,
        y_test_reg
    )
    resultado['Alpha'] = alpha
    resultados_lasso.append(resultado)

tabla_lasso = pd.DataFrame(resultados_lasso).sort_values('MAE').reset_index(drop=True)
display(tabla_lasso[['Modelo', 'MAE', 'RMSE', 'R2', 'MAE_interpretable', 'RMSE_interpretable']])

mejor_alpha_lasso = tabla_lasso.loc[0, 'Alpha']
modelo_lasso_final = modelos_lasso[mejor_alpha_lasso]

print(f"Mejor alpha para Lasso según MAE en test: {mejor_alpha_lasso}")
```

	Modelo	MAE	RMSE	R2	MAE_interpretable	RMSE_interpretable
0	Lasso alpha=0.01	140.429388	253.854748	0.989408	2 min 20 s	4 min 14 s
1	Lasso alpha=0.1	140.814627	253.842788	0.989409	2 min 21 s	4 min 14 s
2	Lasso alpha=1	144.634144	254.680136	0.989339	2 min 25 s	4 min 15 s
3	Lasso alpha=10	177.673946	287.119752	0.986450	2 min 58 s	4 min 47 s
4	Lasso alpha=100	221.644881	328.526358	0.982260	3 min 42 s	5 min 29 s

Mejor alpha para Lasso según MAE en test: 0.01

5.2.8 Interpretación de coeficientes

Una de las ventajas de los GLM es que podemos estudiar sus coeficientes. Como las variables han sido escaladas, los coeficientes no se interpretan en segundos por unidad original, sino como el cambio esperado en el tiempo final cuando una variable aumenta una desviación estándar.

Esto permite comparar la importancia relativa de las variables dentro de cada modelo. Los coeficientes positivos aumentan la predicción de tiempo final y los negativos la reducen.

```
variables_regresion_modelo = X_train_reg_prep.columns

coeficientes_glm = pd.DataFrame({
    'Variable': variables_regresion_modelo,
    'Lineal': modelo_regresion_lineal.coef_,
    'Ridge': modelo_ridge_final.coef_,
    'Lasso': modelo_lasso_final.coef_
})

coeficientes_glm['abs_lasso'] = coeficientes_glm['Lasso'].abs()

display(
    coeficientes_glm
    .sort_values('abs_lasso', ascending=False)
    .drop(columns='abs_lasso')
    .head(12)
)
```

	Variable	Lineal	Ridge	Lasso
8	35k	4025.667627	4025.490154	4023.305867
7	30k	-1326.373114	-1326.077823	-1320.619051
4	20k	-393.110310	-392.528345	-304.873336
5	half	390.308089	389.697613	297.561227
6	25k	-252.133974	-252.199627	-248.663766
1	5k	40.663869	40.662917	38.696002
12	is_Female	-30.534056	-30.535222	-30.555873
9	temp_mean_c	12.105176	12.105204	14.530835
2	10k	-14.779124	-14.775172	-12.185244
0	age	6.956311	6.957113	7.007082
13	is_USA	6.566792	6.566846	6.533043
3	15k	-0.626758	-0.654389	-3.618728

Los coeficientes obtenidos muestran un patrón muy claro: la variable dominante es 35k. En los tres modelos su coeficiente está alrededor de 4025 segundos, muy por encima del resto de variables. Como las variables predictoras han sido escaladas, esto significa que un aumento de una desviación típica en el tiempo de paso por el kilómetro 35 se asocia con un aumento muy grande del tiempo final estimado. Deportivamente tiene todo el sentido: cuanto más tarde llega un corredor al 35k, más tarde llegará normalmente a meta.

Ahora bien, hay que tener cuidado al interpretar los signos del resto de parciales. Variables como 20k, half, 25k, 30k y 35k son tiempos acumulados, por lo que están fuertemente correlacionadas entre sí. Por eso aparecen coeficientes aparentemente contraintuitivos, como 30k negativo o 20k negativo. Esto no significa que pasar más lento por el 30k mejore el tiempo final. Significa que, **manteniendo constante el tiempo de paso por el 35k y el resto de variables**, un mayor tiempo en 30k implica un tramo 30k-35k relativamente más rápido. El modelo está capturando diferencias de ritmo entre parciales acumulados, no efectos aislados simples.

El mismo razonamiento aplica a 20k y half: son dos puntos muy cercanos de la carrera y tienen información casi duplicada. Que uno salga negativo y otro positivo es una señal clásica de **multicolinealidad**. El modelo predice bien, pero los coeficientes individuales de variables muy correlacionadas deben interpretarse como efectos conjuntos y condicionales, no como reglas deportivas independientes.

También se observa que 25k, 10k y 15k tienen pesos menores que 35k y 30k. Esto confirma que, para predecir el tiempo final desde el kilómetro 35, los parciales más cercanos al punto de predicción contienen la mayor parte de la información. 5k tiene un coeficiente positivo pequeño, lo que sugiere que el ritmo inicial aporta algo de información, pero mucho menos que el estado del corredor al final de la carrera.

Las variables no temporales tienen pesos bastante reducidos. `is_Female` aparece con un coeficiente cercano a -30 segundos: manteniendo constantes los parciales y el resto de variables, el modelo estima ligeramente menos tiempo final para mujeres. Este efecto es pequeño comparado con los miles de segundos asociados a 35k, por lo que no debe sobredimensionarse. `temp_mean_c`, `age` e `is_USA` tienen coeficientes positivos modestos, indicando pequeños incrementos esperados del tiempo final al aumentar esas variables, siempre bajo la condición de mantener constantes los parciales.

Por último, la similitud entre Regresión Lineal, Ridge y Lasso indica que la señal principal es muy estable. Ridge apenas cambia los coeficientes respecto a la Regresión Lineal, y Lasso reduce algo más algunos pesos, especialmente en variables redundantes, pero mantiene la misma estructura general. Esto sugiere que el modelo está muy guiado por los parciales acumulados, sobre todo por 35k.

5.2.9 Selección del mejor GLM y validación final

Una vez entrenados los modelos y comparados en `test`, seleccionamos el mejor según MAE. Esta decisión tiene sentido porque queremos minimizar el error medio en segundos. No obstante, también observamos RMSE y R^2 :

- Si dos modelos tienen MAE parecido, preferiremos el que tenga menor RMSE, porque comete menos errores grandes.
- Si MAE y RMSE son similares, el R^2 ayuda a saber cuál explica mejor la variabilidad global del tiempo final.
- La validación final se hace una sola vez sobre `validation`, para estimar cómo se comportaría el modelo elegido ante datos no usados durante la comparación.

```
resultados_glm = pd.DataFrame([
    resultado_lineal,
    tabla_ridge.iloc[0].to_dict(),
    tabla_lasso.iloc[0].to_dict()
])
```



```

])

resultados_glm = resultados_glm.sort_values(['MAE', 'RMSE'], ascending=True).reset_index(drop=True)

print("=== COMPARACIÓN DE MODELOS GLM EN TEST ===")
display(resultados_glm[['Modelo', 'MAE', 'RMSE', 'R2', 'MAE_interpretable', 'RMSE_interpretable']])

mejor_glm = resultados_glm.loc[0, 'Modelo']

if mejor_glm == 'Regresión Lineal':
    modelo_glm_final = modelo_regresion_lineal
elif 'Ridge' in mejor_glm:
    modelo_glm_final = modelo_ridge_final
else:
    modelo_glm_final = modelo_lasso_final

predicciones_val_reg = modelo_glm_final.predict(X_val_reg_scaled)

mae_val = mean_absolute_error(y_val_reg, predicciones_val_reg)
rmse_val = np.sqrt(mean_squared_error(y_val_reg, predicciones_val_reg))
r2_val = r2_score(y_val_reg, predicciones_val_reg)

print(f"\nModelo GLM seleccionado: {mejor_glm}")
print("\n=== EVALUACIÓN FINAL EN VALIDACIÓN ===")
print(f"MAE:    {mae_val:.2f} segundos ({segundos_a_min_seg(mae_val)})")
print(f"RMSE:   {rmse_val:.2f} segundos ({segundos_a_min_seg(rmse_val)})")
print(f"R2:     {r2_val:.4f}")

```

=== COMPARACIÓN DE MODELOS GLM EN TEST ===

	Modelo	MAE	RMSE	R2	MAE_interpretable	RMSE_interpretable
0	Regresión Lineal	140.393223	253.857743	0.989407	2 min 20 s	4 min 14 s
1	Ridge alpha=0.01	140.395199	253.857234	0.989408	2 min 20 s	4 min 14 s
2	Lasso alpha=0.01	140.429388	253.854748	0.989408	2 min 20 s	4 min 14 s

Modelo GLM seleccionado: Regresión Lineal

=== EVALUACIÓN FINAL EN VALIDACIÓN ===

MAE: 139.81 segundos (2 min 20 s)
 RMSE: 262.58 segundos (4 min 23 s)
 R2: 0.9888

5.2.10 Análisis de errores en validación

Además de las métricas agregadas, es recomendable revisar los errores del modelo final. Para ello analizamos los **residuos**, definidos como:

$$\text{Residuo} = y - \hat{y}$$

Si el residuo es positivo, el corredor tardó más de lo que el modelo predijo; es decir, el modelo fue optimista. Si el residuo es negativo, el modelo predijo un tiempo más lento que el tiempo real.

También calculamos el porcentaje de corredores cuya predicción cae dentro de umbrales prácticos de error: 1, 2 y 5 minutos. Esta lectura es muy útil en contexto deportivo porque traduce el error estadístico a una escala fácil de interpretar.

Además, añadimos dos diagnósticos propios de modelos lineales:

- **Residuos frente a valores predichos:** permite detectar si el modelo se equivoca más en corredores rápidos o lentos. Idealmente, los residuos deberían dispersarse alrededor de 0 sin formar un patrón claro.
- **Gráfico Q-Q:** permite comprobar si los residuos se aproximan razonablemente a una distribución normal. Esto es relevante para la interpretación estadística del modelo, aunque para predicción pura es más importante que los errores sean pequeños y estables.

```
df_errores_validacion = pd.DataFrame({
    'Tiempo_real': y_val_reg.to_numpy(),
    'Tiempo_predicho': predicciones_val_reg
})

df_errores_validacion['Residuo'] = (
    df_errores_validacion['Tiempo_real'] - df_errores_validacion['Tiempo_predicho']
)
df_errores_validacion['Error_absoluto'] = df_errores_validacion['Residuo'].abs()

resumen_umbral_error = pd.DataFrame({
    'Umbral': ['<= 1 minuto', '<= 2 minutos', '<= 5 minutos'],
    'Porcentaje de predicciones': [
        (df_errores_validacion['Error_absoluto'] <= 60).mean(),
        (df_errores_validacion['Error_absoluto'] <= 120).mean(),
        (df_errores_validacion['Error_absoluto'] <= 300).mean()
    ]
})

resumen_umbral_error['Porcentaje de predicciones'] = (
    resumen_umbral_error['Porcentaje de predicciones'].map(lambda x: f"{x:.2%}")
)

display(resumen_umbral_error)

fig, axes = plt.subplots(2, 2, figsize=(14, 10))

sns.scatterplot(
    x='Tiempo_real',
    y='Tiempo_predicho',
```

```

    data=df_errores_validacion,
    alpha=0.35,
    ax=axes[0, 0]
)

limite_min = min(
    df_errores_validacion['Tiempo_real'].min(),
    df_errores_validacion['Tiempo_predicho'].min()
)
limite_max = max(
    df_errores_validacion['Tiempo_real'].max(),
    df_errores_validacion['Tiempo_predicho'].max()
)

axes[0, 0].plot([limite_min, limite_max], [limite_min, limite_max], color='red', linestyle='--')
axes[0, 0].set_title('Tiempo real vs tiempo predicho')
axes[0, 0].set_xlabel('Tiempo real (segundos)')
axes[0, 0].set_ylabel('Tiempo predicho (segundos)')

sns.histplot(df_errores_validacion['Residuo'], bins=40, kde=True, ax=axes[0, 1])
axes[0, 1].axvline(0, color='red', linestyle='--')
axes[0, 1].set_title('Distribución de residuos')
axes[0, 1].set_xlabel('Residuo en segundos')
axes[0, 1].set_ylabel('Frecuencia')

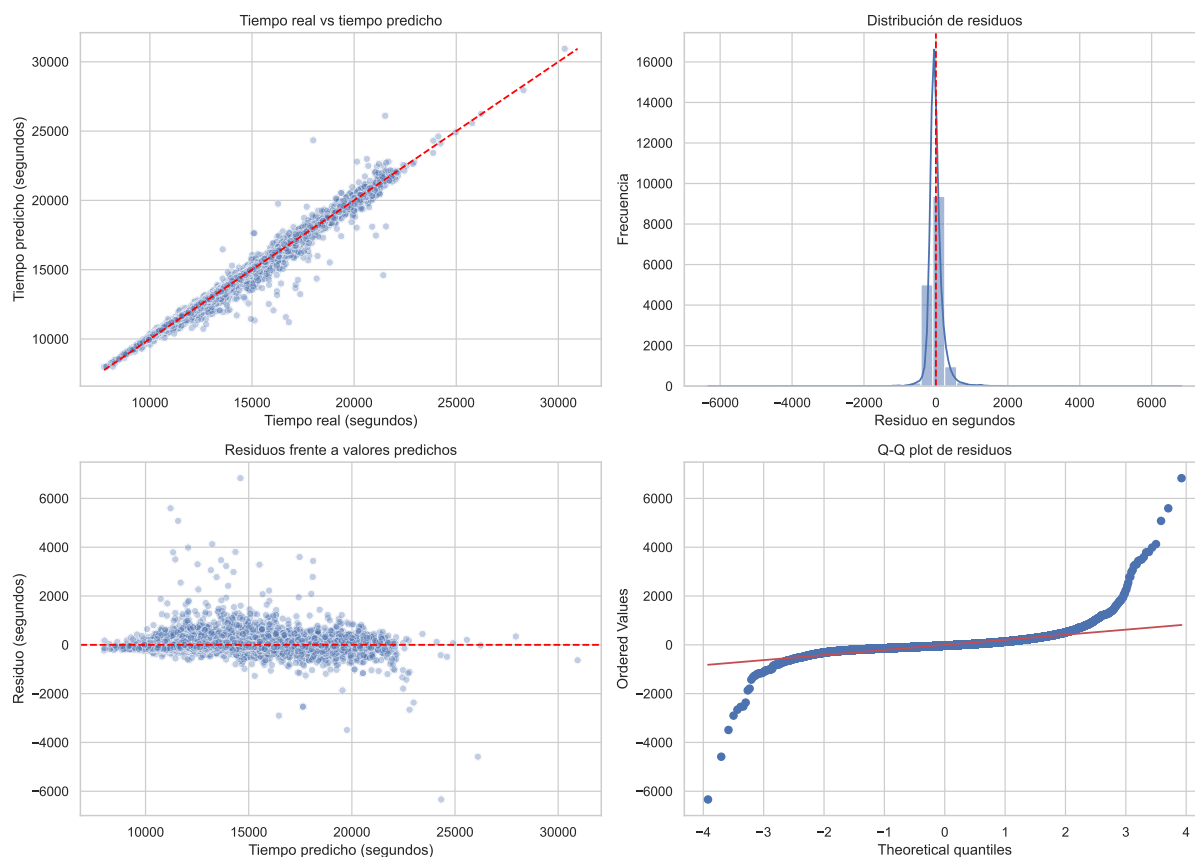
sns.scatterplot(
    x='Tiempo_predicho',
    y='Residuo',
    data=df_errores_validacion,
    alpha=0.35,
    ax=axes[1, 0]
)
axes[1, 0].axhline(0, color='red', linestyle='--')
axes[1, 0].set_title('Residuos frente a valores predichos')
axes[1, 0].set_xlabel('Tiempo predicho (segundos)')
axes[1, 0].set_ylabel('Residuo (segundos)')

stats.probplot(df_errores_validacion['Residuo'], dist='norm', plot=axes[1, 1])
axes[1, 1].set_title('Q-Q plot de residuos')

plt.tight_layout()
plt.show()

```

	Umbral	Porcentaje de predicciones
0	<= 1 minuto	33.51%
1	<= 2 minutos	60.95%
2	<= 5 minutos	91.54%



La primera gráfica, **tiempo real frente a tiempo predicho**, muestra un ajuste global muy bueno: la mayoría de puntos se concentra cerca de la diagonal roja. Esto indica que el modelo reproduce correctamente la relación principal entre el paso por el 35k y el tiempo final. Sin embargo, a medida que aumentan los tiempos finales, la nube se abre un poco más. Es decir, el modelo es más preciso para corredores concentrados en la zona central de la distribución y algo menos estable en corredores muy lentos o con comportamientos de carrera más extremos.

La **distribución de residuos** está muy centrada en 0, lo cual es positivo: en promedio, el modelo no parece tener un sesgo fuerte sistemático. Aun así, la distribución es muy apuntada y con colas largas. Esto significa que la mayoría de corredores se predice con errores pequeños, pero existen algunos casos donde el error es muy elevado. En el contexto de una maratón, estos casos pueden corresponder a atletas que sufren una caída grande de rendimiento tras el 35k, paran, se lesionan, o hacen un último tramo muy distinto al patrón general.

En la gráfica de **residuos frente a valores predichos** se observa que los residuos no forman una nube perfectamente homogénea. Hay más dispersión en la zona media-alta de tiempos predichos y aparecen algunos residuos extremos, tanto positivos como negativos. Recordemos que el residuo se ha definido como **tiempo real - tiempo predicho**: un residuo positivo indica que el modelo fue optimista y predijo un tiempo demasiado rápido; un residuo negativo indica que el modelo fue pesimista y predijo un tiempo demasiado lento. La presencia de residuos positivos altos es especialmente interesante: son corredores que acabaron bastante peor de lo esperado según su paso por el 35k.

El **Q-Q plot** confirma esta lectura. La parte central de los residuos sigue razonablemente bien la línea teórica, pero las colas se separan mucho, sobre todo en los extremos. Por tanto, los residuos no son perfectamente normales: hay colas pesadas y outliers. Esto no invalida el

modelo como herramienta predictiva, especialmente si el MAE y el RMSE son buenos, pero sí nos avisa de que la Regresión Lineal tiene dificultades para capturar comportamientos extremos. Para mejorar esa parte podrían probarse variables de *pacing* entre tramos, transformaciones del *target* o modelos no lineales como árboles de regresión, Random Forest Regressor o XGBoost.

En conclusión, la comparación entre Regresión Lineal, Ridge y Lasso permite valorar tres enfoques lineales con distinto grado de regularización. La Regresión Lineal sirve como referencia interpretable; Ridge suele ser una opción robusta cuando hay variables muy correlacionadas, como ocurre con los parciales acumulados; y Lasso añade la posibilidad de reducir el peso de variables redundantes hasta dejarlas fuera del modelo. La decisión final se basa en el error sobre **test** y se confirma en **validation**, evitando elegir un modelo solo porque se ajusta bien al conjunto de entrenamiento.

6 Discussion/Conclusiones

Aquí irá la discusion